Université Polytechnique HAUTS-DE-FRANCE

INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES HAUTS-DE-FRANCE

Cours :

Le Langage C

Introduction



(C) Achille Braquelaire

R. Tomczak

SOMMAIRE

Table des matières

Pourquoi ce cours ? ---------- 9

Présentation du Module ---------- 9

1. Les compétences et savoir-faire du module ---------- 9
   a. Définition compétence ---------- 9
   b. Définition niveaux d'acquisition – degré d'approfondissement ---------- 9
   c. Savoir : Développement logiciel en langage C ---------- 9
   d. Compétence : développer un module logiciel ---------- 10
   e. Compétence : Tester et valider un module logiciel ---------- 10
2. Utilisation de l'IA ---------- 11
   a. Interdit ? ---------- 11
   b. Règles des 3R ---------- 11
   c. En programmation informatique ---------- 11
3. Test rapide du code ---------- 11
4. IDE ---------- 11

_____ ----------12

CHAPITRE N°1 : ELEMENTS DE BASE ET COMPARAISON AVEC PYTHON 12

_____ ----------12

I. Présentation de la première partie ----------12
   1. Objectifs ---------- 12
   2. Différence Langage C et Python ---------- 12

II. Présentation du langage C ----------13
   1. Historique du langage C ---------- 13
   2. Avantages: ---------- 13
   3. Inconvénients: ---------- 14
   4. Utilisations : ---------- 14
   5. Comparaison entre le langage C et Python ---------- 15
      a. Langage C : ---------- 15
      b. Python : ---------- 15
      c. C vs Python ---------- 15
   6. ***Terminologie*** ---------- 16
      a. **Langages compilés** ---------- 16
      b. **Langages interprétés** ---------- 16
      c. **Compilé Vs Interprété** ---------- 16

d. **Langages semi-interprétés ou semi-compilés** ---- 17
7. Conclusion: ---- 17

III. Le programme principal ----17
1. main() ---- 17
a. Exemple 1 ---- 17
b. Exemple 2 ---- 18
2. ***Chaîne de compilation d'un programme en langage C :*** ---- 18
a. Explication ---- 18
b. Chaine de compilation, langage d'assembleur et langage machine ---- 19

IV. Les variables ----20
1. Les types de données : ---- 20
a. Python ---- 20
b. Langage C ---- 20
2. Déclaration et affectation de variables. ---- 20
a. En Python : lors de la déclaration ---- 21
b. En langage C : obligatoire ---- 21
3. Exemples ---- 21

V. Intéragir avec l'utilisateur ----22
1. Afficher ---- 22
a. Python : print sans le 'f' ---- 22
b. Langage C : printf ---- 22
2. Lire ---- 22
a. Python : input ---- 22
b. C : Lire avec scanf ---- 22
3. Exemples ---- 24

VI. Les branchements conditionnels ----24
1. Si ... alors…(sinon) ---- 24
a. Syntaxe Python ---- 24
b. Syntaxe Langage C ---- 25
c. Exemple ---- 27
2. Selon … cas … ---- 27
a. Syntaxe langage C ---- 27
b. Python : n'existe pas ---- 27

VII. Commentaires ----29
1. Pourquoi commenter un programme informatique ---- 29
2. Les commentaires ---- 29

a. Python ---- 29
b. Langage C ---- 29

VIII. les bibliothèques ----29
1. A quoi servent les bibliothèques ---- 29
2. Fonctionnement des include en langage C ---- 30
3. Les différences entre #include du langage C et import de Python ---- 30
4. Les bibliothèques (ou Library en anglais) ---- 30
a. Python : import ---- 30
b. Langage C : include ---- 31

IX. Les structures de contrôle ----31
1. A quoi sert la boucle pour ---- 31
2. La boucle for pour répéter un certains nombre de fois ---- 31
a. Python : toujours for ---- 32
b. Langage C : for ---- 32
c. Exemple simple ---- 33
3. Pourquoi utiliser un while ? ---- 34
4. While pour répéter en respectant une condition ---- 34
a. En python ---- 34
b. Exemple en Langage C ---- 34
c. Exemples ---- 35
5. La boucle faire … tant que : uniquement en C ---- 35
a. Exemple ---- 35
b. Remarque ---- 35

X. Les fonctions ----36
1. Le concept de fonction ---- 36
2. Les questions qu'il faut se poser avant d'écrire une fonction ---- 36
3. Les fonctions en Python ---- 37
a. Exemple commenté ---- 37
b. Fonction sans paramètre ne retournant pas de valeur ---- 37
c. Fonction sans paramètre, retournant une valeur ---- 37
d. Deux paramètres sans retour de valeur ---- 38
e. Plusieurs paramètres et retournant une valeur ---- 38
f. Les fonctions sont appelées ---- 38
4. Les fonctions en Langage C ---- 39
a. Exemple commenté ---- 39
b. Fonction sans paramètre ne retournant pas de valeur Exemples ---- 39
c. Fonction sans paramètre, retournant une valeur ---- 39
d. Deux paramètres sans retour de valeur ---- 40

e. Plusieurs paramètres et retournant une valeur ---- 40
f. Les fonctions sont regroupées dans le *main* ---- 40

XI. Répartir les fonctions dans des fichiers ----40
a. Création de fichier d'entête ---- 40
b. Création d'un fichier .c par type de fonction ---- 40
c. Appel dans le programme principal ---- 41
d. Compilation ---- 41
2. Comparaison ---- 42
---43
CHAPITRE N°2 : TABLEAUX, CHAINES ET OPERATEURS --------43
---43
I. Les tableaux ----43
1. Les tableaux à une dimension ---- 43
a. Exemples de déclarations ---- 43
b. Initialisation ---- 43
c. Remarque ---- 43
d. Exemple d'utilisation moyenne de 20 notes ---- 43
2. Les tableaux à plusieurs dimensions ---- 44
a. Déclarations – exemples ---- 44
b. Accès ---- 44
c. Représentation graphique ---- 45
d. Initialisation ---- 45
a. Exemples ---- 46
II. LES CHAINES DE CARACTERES ----46
1. Déclaration ---- 46
2. Ajout du caractère nul ---- 46
3. Plusieurs dimensions = liste de chaînes ---- 47
4. Fonctions de manipulations ---- 47
5. Exemple d'utilisation : une chaîne=un tableau ---- 47
6. Utilisation de tableau de char à la place des int ---- 48
III. LES OPERATEURS ----48
1. Définition ---- 48
2. Exemple ---- 49
---51
CHAPITRE N°3 : LECTURE ET ECRITURE DANS UN FICHIER ---51
---51
I. Lecture de fichiers ----51

1. Ouverture d'un fichier ---- 51
2. Lecture de caractères ---- 51
3. Lecture de lignes ---- 51
4. Lecture de blocs de données ---- 51
5. Lecture formatée : utilisation de fscanf ---- 52
6. Fermeture d'un fichier ---- 52
7. Exemples de code ---- 52
   a. En utilisant fgetc ---- 52
   b. En utilisant fgets ---- 52
   c. Lecture d'une chaine et d'un entier avec fscanf ---- 53

II. Ecriture dans des fichiers ---- 53

1. Ouverture d'un fichier en écriture ---- 53
2. Écriture de caractères ---- 54
3. Écriture de chaînes de caractères ---- 54
4. Écriture de données binaires ---- 54
5. Écriture formatée : utilisation de fprintf ---- 54
6. Fermeture d'un fichier ---- 54
7. Exemples ---- 54
   a. Avec uniquement fputs ---- 54
   b. Avec fputs et putc ---- 54
   c. fputs fputc et fwrite ---- 55
   d. Même code avec uniquement printf ---- 55
   e. Ecriture d'une chaine et d'un entire en même temps à l'aide de fprintf ---- 56
8. Conclusion ---- 56

III. Manipulation de fichiers ---- 56

1. Pour lire ou écrire dans un fichier quand faut-il utiliser "wr" ou "rb" ? ---- 57
2. Renommer un fichier : ---- 58
3. Supprimer un fichier : ---- 58
4. Tester l'existence d'un fichier : ---- 58
5. Déplacer un fichier : ---- 59
6. Copier un fichier ---- 59

_____ ---- 61

_____ ---- 61

IV. La création de nouveaux types de variables ---- 62

1. Présentation ---- 62
2. Exemple ---- 62
3. Rappel : ---- 62

V. Le type énumération ---- 62

1. Objectif et exemple ---- 62

2. Avec typedef ---------- 63
VI. Les structures ----------63
1. Introduction ---------- 63
a. Définition d'une structure ---------- 63
b. Pourquoi utiliser des structures ? ---------- 64
c. Syntaxe d'une structure ---------- 64
2. Déclaration et utilisation des structures ---------- 64
a. Déclaration d'une structure ---------- 64
b. Initialisation d'une structure ---------- 64
c. Accès aux membres d'une structure ---------- 65
3. Questions à se poser ---------- 65
a. Quel est le but de la structure ? ---------- 65
b. Quels types de données la structure doit-elle contenir ? ---------- 65
c. Quelles sont les relations entre les données de la structure ? ---------- 65
d. La structure doit-elle être compatible avec d'autres structures ou fonctions du programme ?65
e. Dois-je créer des fonctions spécifiques pour manipuler cette structure ? ---------- 65
4. Structures imbriquées et pointeurs vers des structure ---------- 66
a. Déclaration de structures imbriquées ---------- 66
b. Accès aux membres de structures imbriquées ---------- 66
5. Tableaux de structures ---------- 66
a. Déclaration d'un tableau de structures ---------- 66
b. Accès aux éléments d'un tableau de structures ---------- 66
c. Exemples d'utilisation de tableaux de structures ---------- 67
_____ ----------68
_____ ----------68
I. Introduction ----------68
II. Les variables locales et globales ----------68
1. Les variables et la mémoire ---------- 68
2. Les variables locales ---------- 69
3. Les variables globales ---------- 70
a. Exemple de variable globale ---------- 70
b. Deux variables portant le même nom ---------- 71
c. Trois variables avec le même nom ---------- 72
d. Dernier exemple : ---------- 73
e. Pour ne pas utiliser de variables globales ? ---------- 74
III. Pointeurs et adresses ----------74
1. Adresses ---------- 74
a. Définition d'une adresse ---------- 74
b. Exemple avec les tableaux ---------- 74

2. Pointeurs ---------- 75
a. Définition et déclaration d'un pointeur ---------- 75
b. Initialisation d'un pointeur en langage C ---------- 75
c. Utilisation ---------- 76
d. Affichage de l'adresse ---------- 76
e. Si le pointeur n'est pas initialisé ? ---------- 76
IV. Utilisation des pointeurs ---------- 77
1. Avec les tableaux ---------- 77
a. declaration ---------- 77
b. parcours ---------- 77
c. Accès direct ---------- 77
2. Les opérations arithmétiques sur les pointeurs avec les tableaux ---------- 78
a. Addition ---------- 78
b. la soustraction ---------- 78
c. La comparaison ---------- 78
d. Conclusion ---------- 78
3. A. Pointeurs et fonctions ---------- 78
a. Rappel passage par valeur ---------- 79
b. Passage de pointeurs en paramètres de fonctions ---------- 79
4. Pointeur et tableaux à une dimension ---------- 80
a. Tableau à une dimension première version ---------- 80
b. D'autres version de déclaration de fonction possible ---------- 80
c. D'autres version du code de la fonction : ---------- 81
d. Récapitulatif ---------- 81
e. Nombre d'éléments ---------- 82
5. Tableaux à plusieurs dimensions ---------- 82
6. Passage de structure ---------- 83
V. Allocation dynamique de mémoire ---------- 84
1. Présentation de malloc() ---------- 84
2. Exemple d'utilisation de malloc() pour allouer un tableau d'entiers ---------- 84
3. Utilisation de malloc() pour allouer de la mémoire à une structure ---------- 85

POURQUOI CE COURS ?

Bienvenue dans votre premier cours de langage C.

Vous vous demandez peut-être pourquoi il est important d'apprendre le C, un langage de programmation qui existe depuis plusieurs décennies. La réponse est simple : le C est un pilier fondamental de la programmation. En maîtrisant le C, vous acquérez des compétences essentielles qui vous serviront tout au long de votre carrière en informatique.

Le langage C est bien plus qu'un simple langage de programmation. Il sert de fondation à de nombreux langages modernes comme PHP, Java, JavaScript, C++, et bien d'autres. Il est vrai que le C peut sembler moins accessible que des langages comme Python, nécessitant rigueur et attention (qui n'a jamais oublié ce fameux point-virgule en fin de ligne ?). Cependant, la bonne nouvelle, c'est que la syntaxe du C est relativement épurée, avec peu de concepts à maîtriser pour débuter.

En apprenant le C, vous ne vous contentez pas d'apprendre à coder ; vous apprenez à penser comme un informaticien. Le C vous initie à la discipline, car il exige une compréhension claire de la logique et de la structure. C'est un langage de bas niveau qui, malgré une gestion de la mémoire parfois complexe, offre une perspective précieuse sur le fonctionnement interne des programmes informatiques.

Je prends souvent la comparaison avec des langages : prenons l'exemple de l'anglais, que l'on peut comparer à Python, et de l'allemand, que l'on peut assimiler au C. Avec l'anglais, on peut se débrouiller avec quelques mots de base, mais pour l'approfondir, il faut beaucoup d'effort et de pratique. C'est similaire à Python, où l'on peut rapidement créer des programmes simples, mais où la maîtrise de concepts plus avancés demande du temps et des efforts.

En revanche, l'allemand, comme le C, exige une compréhension rigoureuse de la grammaire et de la syntaxe dès le départ. Une fois ces bases maîtrisées, vous pouvez progressivement enrichir votre vocabulaire et devenir capable de tenir une conversation complexe. De la même manière, en apprenant le C, en comprenant parfaitement sa syntaxe et ses structures de base, vous vous donnez les moyens de construire des programmes plus élaborés et puissants.

Pour conclure, le C est omniprésent dans le monde de la technologie. Il est la base de systèmes d'exploitation comme Linux, ainsi que de nombreuses technologies embarquées et systèmes critiques. En apprenant le C, vous ouvrez la porte à une compréhension profonde du fonctionnement réel des ordinateurs et des logiciels.

PRESENTATION DU MODULE

1. Les compétences et savoir-faire du module

a. Définition compétence

Est une combinaison de connaissances, SF, expériences et comportements

b. Définition niveaux d'acquisition – degré d'approfondissement

c. Savoir : Développement logiciel en langage C

Principe de base

- Organisation des fichiers dans un projet logiciel ; chaîne de développement (préprocesseur, compilateur, éditeur de lien, etc.), **niveau 3**

- Représentation et codage des informations : bases de calcul (2,10,16), réels, caractères, etc. **niveau 4**
- Organisation des programmes : point d'entrée, prototypes, fonctions, paramètres, valeur de retour **niveau 3**

Algorithme

- Structures fondamentales : enchaînements, alternatives, itérations, etc. **niveau 4**
- Manipulations de texte (chaînes de caractères) **niveau 3**
- Algorithmes de tri/de recherche **niveau 2**
- Modèle canonique de gestion d'E/S : ouvrir, lire, écrire, fermer **niveau 3**
- Bibliothèque standard (ANSI C) **niveau 2**

Techniques avancées

- Gestion mémoire : adresse/valeur, pointeurs, et allocations dynamique (pile/tas), etc **niveau 3**
- Variables ; durée de vie, visibilité **niveau 4**
- Types dérivés : tableaux, énumérations, enregistrements **niveau 3**
- Développement de programmes « console » avec gestion des arguments de la ligne de commande **niveau 3**

d. Compétence : développer un module logiciel

Une compétence est une combinaison de savoirs et savoir-faire.

Savoirs

Développement logiciel en langage C

Savoirs Faire SF – résultats attendus

Savoir-Faire	Attendus
Identifier et éliminer les points critiques.	Les erreurs sont identifiées et le module est débuggé. Un rapport de test est rédigé. La documentation est mise à jour.
S'assurer de la robustesse d'un programme.	Le rapport de test est rédigé.
Documenter les interfaces.	La documentation est à jour et conforme aux interfaces
S'assurer de la conformité du logiciel avec le cahier des charges	Le cahier de recette du module est validé.

e. Compétence : Tester et valider un module logiciel

Savoir

Développement logiciel en langage C

Savoir-Faire

Savoir-Faire	Attendus
Éliminer les bugs.	Les erreurs sont identifiées. Les erreurs sont résolues. La documentation est mise à jour
Faire la recette d'un module.	Le cahier de recette est renseigné.
Établir un compte rendu de performance.	Un compte rendu de performance est rédigé

2. Utilisation de l'IA

a. Interdit ?

En absence de consignes, les IA ne sont pas interdites, mais fortement déconseillées en TD.

b. Règles des 3R

Voici un des principes que j'utilise avec chatGPT, la règles de 3R :

- **Relire** : l'IA peut faire des erreurs de grammaire, de syntaxe ou il traduit des termes en français qu'on veut garder en anglais
- **Rectifier** : parce que l'IA ne donne pas la bonne solution, mais la plus vraisemblable mais aussi pour donner une touche personnelle au texte
- **Rajouter** : après relecture il est souvent nécessaire d'ajouter une explication complémentaire par vous-même sans avoir à demander à l'IA. Illustrer par exemple avec des schémas

Et surtout : la relecture vous fait comprendre le contenu du texte et à renforce vos connaissances.

c. En programmation informatique

Il faut être un expert pour utiliser le travail de l'IA : le rectifier, gérer les pbs de versions..

3. Test rapide du code

Vous pouvez tester le code en utilisant un compilateur en ligne :

- http://www.tutorialspoint.com/compile_c_online.php
- https://www.codechef.com/ide
- https://replit.com/languages/python3

Le gros avantage c'est qu'il n'y a rien à installer.

4. IDE

Le ou les seuls IDE qui devraient être utilisées sont :

- Visual Studio Community (https://visualstudio.microsoft.com/fr/thank-you-downloading-visual-studio/?sku=Community&channel=Release&version=VS2022&source=VSLandingPage&passive=false&cid=2030)
- VSCodium : https://vscodium.com/
- Atom : http://atom.io/
- Geany : https://www.geany.org/
- Brackets : http://brackets.io/
- Sublime : https://www.sublimetext.com/

CHAPITRE N°1 : ELEMENTS DE BASE ET COMPARAISON AVEC PYTHON

I. PRESENTATION DE LA PREMIERE PARTIE

1. Objectifs

Le principal objectif de ce document est d'écrire rapidement un programme **simple** dans le langage C. Pour atteindre ce but, je partirai d'exemple dans le langage Python.

Plus précisément les compétences de ce cours associé aux TD-TP sont :

- **Développer un module logiciel**
- Tester et valider un module logiciel

Les savoir-faire sont :

- Maîtriser les structures fondamentales : enchaînements, alternatives, itérations, etc.
- Transcrire un algorithme en langage C

Par contre, ce cours n'est pas accès sur l'algorithmie mais bien sur l'implémentation des algorithmes en langage C.

Dans cette optique, le choix a été fait de ne parler que du **strict minimum** et ne pas présenter les spécificités du langage C dans cette première partie. Les approfondissements se feront dans les **autres cours**.

2. Différence Langage C et Python

Le langage C est un « vieux » langage de programmation datant des années 1970. Il est considéré comme généraliste et proche du matériel.
C'est un langage compilé : le code source est transformé en code binaire directement exécutable par la machine.
Sa connaissance est incontournable en informatique.

Python lui date des années 90, également généraliste, il est bien plus facile à apprendre que le C.
Langage interprété son code est exécuté pas à pas.
Sa connaissance est également incontournable.

Visuellement, la première différence, c'est **l'absence de *main*** en début de code, des **points-virgules** à la fin de chaque ligne et des tabulations à la place d'accolades.

II. PRESENTATION DU LANGAGE C

C est un langage de programmation de bas niveau, qui a été développé dans les années 1970 par Dennis Ritchie à Bell Labs. C est un langage généraliste qui peut être utilisé pour de nombreux types de développement, y compris le système, les applications, les jeux, et plus encore.

1. Historique du langage C

L'histoire du langage C débute dans les années 1960, lorsque les ingénieurs de Bell Labs ont commencé à travailler sur un nouveau système d'exploitation appelé Unix. Pour développer ce système, ils avaient besoin d'un langage de programmation qui soit simple, flexible et performant. Dennis Ritchie a été chargé de développer ce langage.

En 1972, Ritchie a publié la première version du langage C, appelé "C profond". Ce langage était basé sur d'autres langages de programmation tels que BCPL et B, mais avait des caractéristiques uniques qui le distinguaient.

Dans les années 1980, C est devenu un des langages de programmation les plus populaires pour le développement de systèmes d'exploitation et d'applications embarquées. De nombreux systèmes d'exploitation importants, tels que Unix, Linux et Windows, ont été développés en utilisant le langage C.

En 1983, Bjarne Stroustrup a développé un nouveau langage appelé C++ qui était basé sur C, mais qui ajoutait de nouvelles fonctionnalités telles que la programmation orientée objet. C++ est devenu un langage très populaire pour le développement d'applications de haute performance.

Malgré la popularité croissante de C++, C est resté un langage important pour de nombreux développeurs. En 1999, la norme C99 a été publiée pour moderniser le langage. Depuis lors, C est resté l'un des langages de programmation les plus populaires pour le développement de systèmes d'exploitation, d'applications embarquées et de bibliothèques pour d'autres langages.

En résumé, le langage C est un langage de programmation important qui a été développé dans les années 1970 pour le développement de systèmes d'exploitation. Depuis lors, il est devenu l'un des langages de programmation les plus populaires pour de nombreux types de développement.

2. Avantages:

Le langage C a plusieurs avantages qui en font un choix populaire pour de nombreux développeurs :

1. **Portabilité** : C est un langage portable qui peut être utilisé sur de nombreux systèmes d'exploitation différents sans modification.
2. **Performances** : C est un langage de bas niveau qui donne aux développeurs un contrôle étendu sur les ressources du système. Il permet d'obtenir des performances élevées pour les applications critiques en termes de temps.
3. **Large écosystème** : C a un écosystème très large avec de nombreuses bibliothèques et outils disponibles pour les développeurs.
4. **Apprentissage facile** : C est un langage de programmation facile à apprendre pour les développeurs débutants. C'est une excellente option pour ceux qui cherchent à apprendre les bases de la programmation.
5. **Interopérabilité** : C peut être utilisé avec d'autres langages de programmation pour créer des applications complexes. C'est souvent utilisé pour écrire des bibliothèques pour d'autres langages.

6. **Stabilité** : C est un langage de programmation stable qui a été utilisé pendant plusieurs décennies et a été largement testé. Il est peu probable qu'il disparaisse ou soit remplacé par un autre langage.

En conclusion, le langage C offre une combinaison de portabilité, de performances, de simplicité, de stabilité et d'un large écosystème, qui en font un choix populaire pour de nombreux développeurs

3. Inconvénients:

Le langage C a également certains inconvénients qui peuvent être considérés comme des défis pour les développeurs :

1. **Sécurité** : C est un langage de bas niveau qui donne aux développeurs un grand contrôle sur les ressources du système, ce qui peut entraîner des problèmes de sécurité si les développeurs ne sont pas attentifs.
2. **Manque de fonctionnalités de haut niveau** : C est un langage de bas niveau et ne fournit pas les fonctionnalités de haut niveau telles que la gestion automatique de la mémoire, la gestion des exceptions, la programmation orientée objet, etc. qui sont couramment disponibles dans d'autres langages.
3. **Difficile à lire et à maintenir** : C peut être difficile à lire et à maintenir en raison de la syntaxe concise et de la nécessité de gérer manuellement la mémoire.
4. **Debugging** difficile : La déboguage peut être difficile dans C en raison de l'absence de contrôle automatique de la mémoire et de la nécessité de faire attention aux erreurs de pointeur.
5. **Inconsistance des bibliothèques** : Les bibliothèques en C peuvent être incohérentes en termes de style et de qualité, ce qui peut rendre difficile la lecture et le maintien du code.

En conclusion, le langage C a certains inconvénients tels que la sécurité, le manque de fonctionnalités de haut niveau, la difficulté de lecture et de maintenance, la difficulté de déboguage et l'inconsistance des bibliothèques, qui peuvent rendre difficile la programmation en C pour certains développeurs.

4. Utilisations :

Le langage C est utilisé dans de nombreux domaines pour différentes applications, quelques exemples incluent :

1. **Systèmes d'exploitation** : De nombreux systèmes d'exploitation tels que Windows, Unix, Linux, etc. sont écrits en C. C est un langage de bas niveau qui peut être utilisé pour accéder directement aux fonctionnalités du système d'exploitation.
2. **Logiciels de développement d'applications** : C est utilisé pour développer des applications telles que les éditeurs de texte, les compresseurs de fichiers, etc.
3. **Jeux vidéo** : C est un langage de programmation de jeu populaire qui peut être utilisé pour développer des jeux pour PC, consoles de jeux et appareils mobiles.
4. **Pilotes** : Les pilotes de périphériques tels que les imprimantes, les disques durs, les cartes graphiques, etc. sont souvent écrits en C en raison de ses capacités à accéder directement aux ressources du système.
5. **Applications scientifiques et techniques** : C est utilisé pour développer des applications scientifiques et techniques telles que les programmes de calcul de formules mathématiques, les simulations, les analyses de données, etc.

En conclusion, le langage C est utilisé dans de nombreux domaines pour diverses applications, y compris les systèmes d'exploitation, les logiciels de développement d'applications, les jeux vidéo, les pilotes et les applications scientifiques et techniques.

5. Comparaison entre le langage C et Python

Le langage C et Python sont deux langages de programmation très différents qui sont utilisés pour des applications différentes.

a. Langage C :

- Bas niveau : Le C est un langage de bas niveau qui permet un accès direct aux fonctionnalités du système.
- Performances : Le C offre des performances élevées pour les applications qui nécessitent une rapidité d'exécution.
- Syntaxe : La syntaxe du C est rigoureuse et nécessite une attention particulière aux détails pour éviter les erreurs.
- Utilisation : Le C est souvent utilisé pour les systèmes d'exploitation, les pilotes, les jeux vidéo, les applications scientifiques et techniques.

b. Python :

- Haut niveau : Python est un langage de programmation de haut niveau qui offre une syntaxe simple et intuitive.
- Productivité : Python est considéré comme un langage très productif grâce à ses bibliothèques et frameworks développés.
- Flexibilité : Python est très flexible et peut être utilisé pour de nombreuses applications, y compris les applications web, la science des données, l'IA et l'apprentissage automatique.
- Utilisation : Python est souvent utilisé pour les applications web, la science des données, l'IA et l'apprentissage automatique.

En conclusion, le choix entre le langage C et Python dépend des besoins spécifiques de l'application. Si vous avez besoin de performances élevées, le C peut être le meilleur choix, mais si vous recherchez une productivité accrue et une flexibilité accrue, Python pourrait être un meilleur choix.

c. C vs Python

	Langage C	Python
Bas ou Haut niveau	**Le C est un langage de bas niveau qui permet un accès direct aux fonctionnalités du système.**	**Python est un langage de programmation de haut niveau qui offre une syntaxe simple et intuitive.**
Performance / productivité	**Le C offre des performances élevées pour les applications qui nécessitent une rapidité d'exécution.**	**Python est considéré comme un langage très productif grâce à ses bibliothèques et Framework développés.**
Syntaxe / flexibilité	**: La syntaxe du C est rigoureuse et nécessite une attention particulière aux détails pour éviter les erreurs**	**Python est très flexible et peut être utilisé pour de nombreuses applications, y compris les applications web, la science des données, l'IA et l'apprentissage automatique.**
Utilisation	**Le C est souvent utilisé pour les systèmes d'exploitation, les pilotes, les jeux vidéo, les applications scientifiques et techniques.**	**Python est souvent utilisé pour les applications web, la science des données, l'IA et l'apprentissage automatique**

6. Terminologie

a. Langages compilés

Un langage compilé comme le langage C voit son code source transformé en un fichier exécutable, cad un fichier binaire qui n'est exécutable que pour un type de machine : ainsi le programme qui a été compilé sous Linux, ne fonctionnera pas sous Windows, il faudra donc le re-compiler. Ce processus de compilation permet une meilleure performance en cours d'exécution, mais peut être plus lent et plus difficile à développer en raison de la nécessité de recompiler le code à chaque modification.

L'élément clé est le compilateur, c'est un programme qui va compiler un autre programme : gcc compilateur GNU principalement sous Linux (et Windows) ou MSVC uniquement pour Windows

Les langages compilés sont : C, C++, Pascal ou OCaml

b. Langages interprétés

Les langages interprétés sont exécutés directement sans être compilés en code machine. Cela rend le développement plus rapide et plus flexible, mais peut entraîner une performance en cours d'exécution inférieure.

Le langage Python est le plus connu des langages interprétés.

Un programme Python est exécuté de la manière suivante :

1. Interprétation : Python est un langage interprété, ce qui signifie que le code source est analysé et exécuté ligne par ligne à chaque fois qu'il est exécuté.
2. Chargement de l'interpréteur : Pour exécuter un programme Python, l'interpréteur Python doit être installé sur l'ordinateur. L'interpréteur peut être démarré en ligne de commande ou à partir d'un environnement de développement intégré (IDE).
3. Exécution du code : Lorsque l'interpréteur est en marche, le code Python peut être saisi et exécuté immédiatement. Le programme peut également être enregistré dans un fichier .py et exécuté en utilisant la commande "python nom_du_fichier.py".
4. Sortie des résultats : Une fois le code exécuté, les résultats sont affichés à l'écran. Les résultats peuvent également être enregistrés dans un fichier pour une utilisation ultérieure.

L'exécution d'un programme Python implique l'interprétation du code source par l'interpréteur Python, le chargement de l'interpréteur, l'exécution du code et la sortie des résultats.

A noter que l'interpréteur de Python est un programme écrit en langage C.

Les langages interprétés les plus connus sont : Python, Php, javascript, Ruby

c. Compilé Vs Interprété

Voici un tableau de comparaison entre les langages compilés et interprétés :

Compilé:

- Conversion du code source en code machine directement
- Exécution plus rapide car le code est précompilé
- Code source caché et non modifiable
- Erreurs généralement détectées lors de la compilation

Interprété:

- Exécution du code source ligne par ligne
- Flexibilité pour le développeur de voir les erreurs et de les corriger en temps réel
- Code source facilement modifiable
- Exécution plus lente car nécessite une interprétation à chaque fois

Il convient de noter que les langages peuvent être classés comme compilés ou interprétés en fonction de la façon dont ils sont exécutés, mais certains peuvent être les deux à la fois ou avoir des caractéristiques des deux.

d. Langages semi-interprétés ou semi-compilés

Un langage semi-interprété est un langage de programmation qui n'est pas entièrement compilé ou interprété, mais qui combine les avantages des deux. Il peut être compilé en une forme intermédiaire pour améliorer les performances, puis exécuté par un interpréteur. Cela permet d'avoir un développement plus rapide et une meilleure flexibilité que la compilation complète, tout en conservant une certaine performance en cours d'exécution.

Notez que la distinction entre les langages interprétés et semi-interprétés n'est pas toujours claire et peut varier en fonction de la définition utilisée.

Par exemple le java peut être considéré comme un langage semi-interprété : c'est un langage de programmation orienté objet qui est compilé en code machine portable appelé bytecode, puis exécuté par une machine virtuelle Java.

Mais il peut donc être considéré comme un langage compilé avec une étape intermédiaire d'exécution.

7. Conclusion:

C est un langage de programmation puissant et performant qui est toujours largement utilisé aujourd'hui, malgré sa complexité. Il offre de nombreux avantages pour les développeurs expérimentés, mais peut être difficile à apprendre pour les débutants. Il est important de comprendre les avantages et les inconvénients du langage C avant de décider s'il convient à un projet particulier.

III. LE PROGRAMME PRINCIPAL

1. main()

En C, la fonction `main()` est la fonction principale qui est le point d'entrée du programme. C'est le premier endroit où le code exécutable démarre lorsque le programme est lancé. La déclaration de la fonction `main()` est généralement la première chose que le programmeur écrit lorsqu'il crée un programme en C.

a. Exemple 1

Voici un exemple de programme en C qui affiche "Hello World" avec des commentaires pour expliquer le code :

```
// Inclusion de la bibliothèque standard d'entrée/sortie (input/output)

#include <stdio.h>

// La fonction principale du programme, qui retourne un entier et ne prend pas
d'arguments

int main(void) {
// Affichage du message "Hello World" à l'aide de la fonction printf()
   printf("Hello World");
```

```
    return 0; // Retourne 0 pour indiquer que le programme s'est terminé avec succès
}
```

Remarque : Les commentaires dans le code sont précédés par `//` et sont ignorés par le compilateur. Ils sont utilisés pour expliquer le fonctionnement du code et aider les personnes à comprendre ce qui se passe dans le programme.

La fonction `main()` doit retourner un `int` et peut prendre des arguments en entrée (tels que `int argc, char *argv[]`), qui sont généralement utilisés pour passer des arguments en ligne de commande au programme. Le corps de la fonction `main()` contient tout le code qui doit être exécuté lorsque le programme démarre. Une fois que le code dans la fonction `main()` est terminé, la fonction retourne un entier qui est utilisé pour indiquer le statut de sortie du programme. Un retour de 0 signifie que le programme s'est terminé avec succès, tandis qu'un autre entier peut indiquer une erreur ou un échec.

b. Exemple 2

Voici un exemple de programme en C qui demande le nom de l'utilisateur et affiche "Bonjour + nom" avec des commentaires pour expliquer le code :

```
// Inclusion de la bibliothèque standard d'entrée/sortie (input/output)
#include <stdio.h>
// Inclusion de la bibliothèque standard de chaînes de caractères (strings)
#include <string.h>

// La fonction principale du programme, qui retourne un entier et ne prend pas
d'arguments
int main(void) {
    // Déclaration d'un tableau de caractères pour stocker le nom de l'utilisateur
    char nom[100];

    // Demande le nom de l'utilisateur
    printf("Entrez votre nom : ");
    // Utilise la fonction scanf() pour lire la saisie de l'utilisateur et la stocker dans
le tableau nom
    scanf("%s", nom);

    // Affiche le message de bienvenue en utilisant le nom saisi
    printf("Bonjour %s\n", nom);

    // Retourne 0 pour indiquer que le programme s'est terminé avec succès
    return 0;
}
```

Dans ce programme, nous utilisons la fonction `scanf()` pour lire la saisie de l'utilisateur et la stocker dans le tableau de caractères `nom`. Nous utilisons ensuite `printf()` pour afficher le message de bienvenue en incluant le nom saisi. La fonction `strlen()` de la bibliothèque `string.h` est utilisée pour déterminer la longueur de la chaîne de caractères (c'est-à-dire le nombre de caractères dans le nom).

2. *Chaîne de compilation d'un programme en langage C :*

a. Explication

La chaîne de compilation d'un programme en langage C est un processus qui transforme le code source en un fichier exécutable. Elle comprend plusieurs étapes :

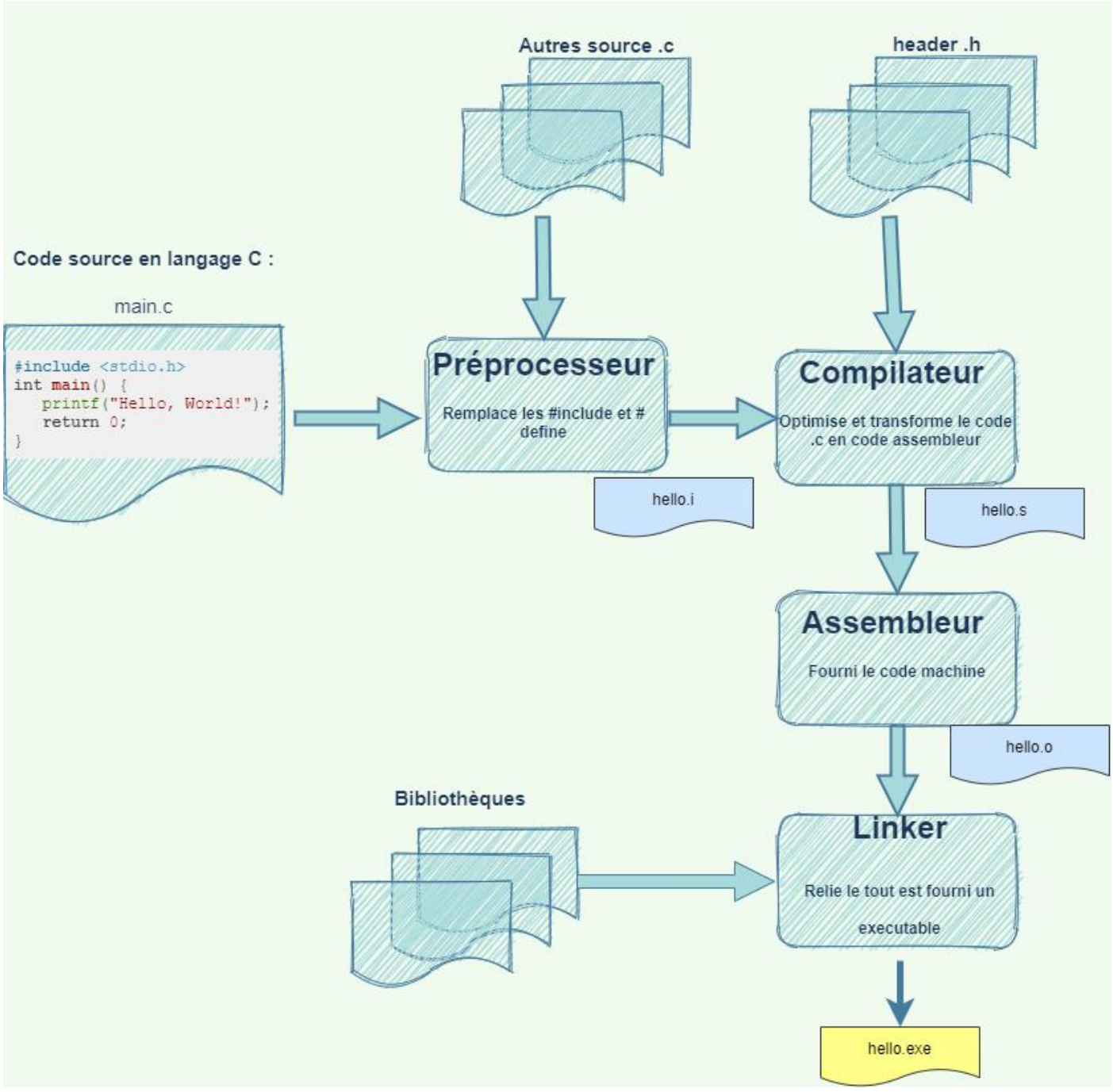
1. Pré-traitement : Le pré-processeur remplace les directives `#include` et `#define` dans le code source par leur contenu respectif.
2. Compilation : Le compilateur traduit le code source en code objet, qui est un code intermédiaire qui peut être compris par le processeur mais qui n'est pas encore exécutable.

3. Edition de liens : L'éditeur de liens combine le code objet avec des bibliothèques externes pour produire un fichier exécutable.
4. Exécution : Le fichier exécutable est finalement exécuté sur le système cible.

En résumé, la chaîne de compilation d'un programme en langage C comprend le pré-traitement, la compilation, l'édition de liens et l'exécution pour produire un fichier exécutable fonctionnel.

b. Chaine de compilation, langage d'assembleur et langage machine

Le compilateur va transformer le code langage C en assembleur puis le programme d'assemblage va traduire le code 'Assembleur' en binaire.



Le langage d'assembleur ou plus communément appelé assembleur, est une suite d'instructions. Par exemple :

Langage C	Assembleur 8086	Langage machine

if ax=0
ax=1 ;
else
{
Ax=2 ;
BX = BX-1 ;
}
	CMP AX,0
	JNZ LABEL
	MOV AX,1
	JMP END
	LABEL :
	MOV AX, 2
	DEC BX
	END:
		0101101101101101
		1011010111111101
		1111110110101010
		1010101110101101
		1011011010010110
		1101110010101101
		1011101110110001
		1101101101011010

IV. LES VARIABLES

1. *Les types de données :*

a. Python

Les types de données de base sont :

Type	Utilisation
int	Entier positif ou négatif
float	Nombre réel à virgule flottante
str	Chaîne de caractère
complex	Nombre complexe
bool	Booléen : True ou False uniquement

b. Langage C

Les principaux types de variables sont :

Type	Nombre d'octets	Plage de valeurs	Utilisation
char	1	-128 (2^7) à 127 (2^7-1)	Représente un seul caractère (code ASCII) et peut être utilisé pour un « petit » entier compris entre -128 et 127.
int	2 (µp 16 bits) 4 (32 bits)	-32768 (-2^{15}) à 32767 (2^{15}-1)	Entier positif ou négatif.
float	4	3.4028235E+38 à -3.4028235E+38	Nombre réel

Remarques :

- Il existe également des chaînes de caractères, mais ce sont des tableaux de caractères (voir *chaînes de caractères*)
- Le terme *unsigned* mis devant le type de la variable signifie *non signé* et représente uniquement les nombres positifs : la plage de valeurs est ainsi doublé.

Par exemple :

unsigned char	**Taille = 1 octet**	**De 0 à 255 (2^8-1)**	**Représente un seul caractère (code ASCII) ou un entier compris entre 0 et 255.**

2. *Déclaration et affectation de variables.*

Afin que votre code soit le plus compréhensible possible, il faut veiller à utiliser des noms de variables pertinents : le nom de votre variable doit indiquer sa fonction.

`aire, perimetre, volume` à la place de `a, b, c`

Exception faite pour les variables d'incrémentation dont les noms sont traditionnellement `i, j ou k`

a. En Python : lors de la déclaration

Il n'y a pas de déclaration de variable, la définition du type de la variable se fait automatiquement lors de l'affectation :

```
x = 3
pi = 3.14159
message = 'Coucou'
```

Il est possible de connaître le type d'une variable par la fonction *type :*

type(x)	<class 'int'>
type(pi)	<class 'float'>
type(message)	<class 'str'>

b. En langage C : obligatoire

Déclaration : une variable doit être obligatoire déclarée avant d'être utilisée :

```
int i ;
```

On peut déclarer plusieurs variables de même type sur la même ligne :

```
float aire, perimetre, volume ;
```

Pour des raisons de lisibilité, il est fortement conseillé de déclarer les variables au début du code.

Affectation : par la suite dans le code, on peut affecter une valeur à la variable.

```
aire = -4.2;
```

Déclaration et affectation : On peut donner à la variable, une valeur initiale.

```
int vit= -4;
char i, j, toto ;
```

3. *Exemples*

Langage C

```c
#include <stdio.h>
#include <string.h> // pour strcpy
int main()
{
    /* Déclaration */
    int i ;
    float rayon ;
    char nom[10]; // 10 caractères maxi
    char ma_lettre; // 1 caractère

    /* Affectation */
    i=0;
    rayon=8.2;
    strcpy(nom,"Nestor");
    ma_lettre = 'd';

    return 0;
}
```

Python

```python
i = 0
rayon = 8.2
nom = "Nestor" # ou nom='Nestor'
OuiNon = True
```

V. INTERAGIR AVEC L'UTILISATEUR

1. Afficher

a. Python : print sans le 'f'

L'affichage peut se faire simplement sans avoir recours à l'écriture formatée.

➢ Pour un texte :

```
print ('Hello World')
print ('Hello' + 'World') #Affiche HelloWorld
print (3 * '*') #Affiche ***
```

➢ Une variable :

```
rayon = 8.2
print (rayon)
```

➢ Texte + variable :

```
print ('rayon = ', rayon)
```

Ou alors convertir la variable en string pour la concaténer avec la chaîne 'rayon' comme ceci :

```
print('rayon = ' + str(rayon) )
```

b. Langage C : printf

La fonction *printf* se trouve dans la bibiothèque standard : `#include <stdio.h>`

Elle permet de réaliser des sorties formatées de messages et/ou de valeurs des variables sous différents formats.Les formats disponibles sont :

- %d : *Entier Décimal*
- %x : *Entier Hexadécimal*
- %u : *Entier Non Signé*
- %c : *Caractère*
- %s : *Chaîne de caractères*
- %f : *Flottant*

2. Lire

a. Python : input

La fonction essentielle est *input* qui lit une chaîne de caractère :

```
print('Entrez un message')
message = input()
print(message)
```

Une écriture équivalente quasi (hormis le saut de ligne) équivalente est :

```
message = input('Entrez un message : ')
print(message)
```

Pour les autres types de variables que *string,* il faut convertir la chaîne :

➢ Pour lire un entier : `i = int(input())`

➢ Un réel : `rayon = float(input())`

b. C : Lire avec scanf

Elle permet de saisir des valeurs de variables formatées à partir du clavier. Comme printf elle est composée d'un format et des identificateurs de variables à saisir.
A la **différence** de printf, le format ne peut contenir de texte, il est juste composé du format des valeurs à saisir.

scanf (format, d'adresses) : permet de saisir les données au clavier

Exemple 1	Exemple 2
`scanf("%d",&a);` `// Attend la saisie d'un entier`	`scanf("%d%d%f",&a,&b,&c);` `/* Attend la saisie de deux entier puis d'un float`

	`Tous seront séparés par des espaces */`

Remarques:

- Le symbole & est obligatoire sauf pour les tableaux
- Les formats disponibles sont les mêmes que pour *printf* :
  - %d : *Entier Décimal*
  - %x : *Entier Hexadécimal*
  - %u : *Entier Non Signé*
  - %c : *Caractère*
  - %s : *Chaîne de caractères*
  - %f : *Flottant*

3. Exemples

Langage C	Python

`#include <stdio.h>`
`int main ()`
`{`
`  /* Déclarations et affectation */`
`  float rayon = 8.2;`
`  int i;`
`  // Chaine de 10 caractères maxi :`
`  char chaine [10];`
 	
`  // Avec un saut de ligne \n :`
`  printf("Hello World \n");`
 	
`  //Affiche HelloWorld`
`  printf("Hello");`
`  printf("World\n");`
`  // But! 3 fois`
`  printf("But! But! But!\n");`
 	
`  printf("%f\n", rayon);`
`  printf("rayon = %f \n", rayon);`
 	
`  printf("Entrez un message :");`
`  // Attention pas de & devant message :`
`  scanf("%s", chaine);`
`  printf("Le message est %s \n", chaine);`
 	
`  printf ("i = ");`
`  scanf ("%d",&i);`
`  printf ("rayon = ");`
`  scanf ("%f",&rayon);`
`  printf ("i = %d \n",i);`
`  printf ("rayon = %f\n",rayon);`
 	
`  return 0;`
`}`
	`print ('Hello World')`
	
	`#Affiche HelloWorld :`
	`#Ces deux lignes affiche HelloWorld0000`
	`print ('Hello' + 'World')`
	`# end = "" -> sans saut de ligne`
	`print ('Hello', end = "");`
	`print ('World');`
	
	`#Affiche But! But! But!`
	`print (3 * 'But! ')`
	
	`rayon = 8.2`
	`print (rayon)`
	`print ('rayon = ', rayon)`
	
	`message = input ('Entrez un message : ')`
	`#Avec + pas de saut de ligne`
	`print ('Le message est ' + chaine)`
	
	`i = int (input ('i =  '))`
	`rayon = float (input ('rayon = '))`
	`print ('i = ' + str (i))`
	`print ('rayon = ' + str (rayon))`

VI. LES BRANCHEMENTS CONDITIONNELS

1. Si … alors…(sinon)

Condition du test : ==, <,>,<=,>=, !=,&&,|| …

a. Syntaxe Python

Python	Explications

`note = 8`
`if note > 10:`
`    print('reçu')`
`    print('bravo !')`
`else:`
`    print('recalé')`
	1. L'instruction if
	2. Le test conditionnel se termine par « : » le signe **deux points.**
	3. A la place des parenthèses du langage C, python utilise **l'indentation** : quatre espaces ou une tabulation, nous préférons le second format
	4. Un bloc d'instruction (une ou plusieurs lignes) dans le cas où la condition est vérifiée
	5. Else optionnelle pour indique le cas où la condition n'est pas vérifiée
	6. A nouveau les deux points
	7. Et un bloc d'instructions

Remarques

- il ne faut pas confondre l'opérateur d'égalité == et celui de l'affection =
- *sinon* est optionnel
- les parenthèses entourant la condition ne sont pas obligatoire (if note>10 ou if (note>10))

Les condition du test sont les mêmes que pour la langage C : ==,<,>,<=,>=, !=, sauf **and**, **or.**

Il est possible Avec else if contracté en elif
Remarquer les **indentations** (elif comme else est au même niveau que le if)

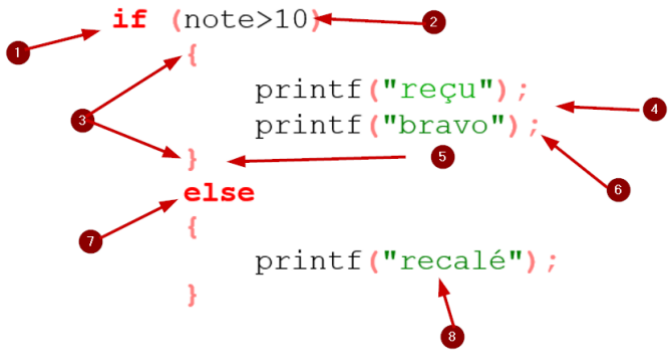
```
x = 'Roger'
if x == 'roger':
    print("manque une majuscule")
elif x == 'Roger ':
    print("un espace en trop")
else:
    print('Pas trouvé!')
```

b. Syntaxe Langage C

Pour faire des choix : l'instruction if

Algo	Langage C

Si a > b
Alors
Afficher "A>B"
Sinon
Afficher "A>B"
FinSi
	`if (a > b)`
	`{`
	`printf("A>B \n");`
	`}`
	`else`
	`{`
	`printf("A<B \n");`
	`}`

Langage C	Explications
	1. L'instruction **if** 2. Le **test conditionnel** peut être entre parenthèse et saut de ligne**. Pas de point virgule à la fin** 3. A la place de l'indentation de Python, le langage C utilise **des parenthèses** : une première ouvrante 4. Un bloc d'instruction (une ou plusieurs lignes) dans le cas où la condition est vérifiée 5. **Une parenthèse** qui ferme le bloc d'instructions 6. Les fins de ligne d'instruction avec des points virgules 7. **Else** optionnelle pour indique le cas où la condition n'est pas vérifiée 8. Et un **bloc** d'instructions

Remarques

- Le code est plus lisible avec des tabulations
- les parenthèses entourant la condition n'est pas obligatoire (if note>10 ou if (note>10))
- il ne faut pas confondre en C, l'opérateur d'égalité == et celui de l'affection =
- *else* (*sinon)* est optionnel,
- si le bloc d'instruction ne contient qu'une seule ligne, il n'est pas obligatoire de l'entourer de parenthèses
- il est possible d'imbriquer des if avec else if

c. Exemple

Langage C	Python

`if (note>10)`
`{`
`printf("reçu");`
`printf("bravo");`
`}`
`else`
`{`
`printf("recalé");`
`}`
	`if note>10:`
	`print("reçu")#une tabulation avant`
	`print("bravo")`
	`else:`
	`print("recalé")`

2. *Selon … cas …*

Cette structure remplace une série de if consécutifs avec l'avantage d'une meilleure lisibilité.

a. Syntaxe langage C

Algo	Langage C

Saisir jour
Selon jour :
Cas 1 : afficher "lundi"
Cas 2 : afficher "mardi"
…
Cas 7 : afficher "dimanche"
	`int jour ;`
	`scanf("%d", &jour);`
	
	`switch (jour) {`
	`case 1 : printf("lundi\n");`
	`break;`
	`case 2 : printf("mardi\n");`
	`break;`
	…
	`case 7 : printf("dimanche\n");`
	`break;`
	`default : printf ("Erreur : le jour doit être compris entre 1 et 7\n");`
	`}`

Remarques sur la syntaxe :

- Tous les cas sont dans une même accolade `{…}`
- Chaque ligne commence par `case x :` (les deux points sans le point-virgule)
- `x` doit être de type entier
- Mais comme 'A' est le code ascii de la lettre A, il est possible d'écrire `case 'A' :`
- Faire attention de bien mettre l'instruction `break`; qui fait sortir du `switch` – `case`, sinon on continue jusque `default` !!
- Si aucune valeur n'existe, les instructions sous `default` sont exécutées

b. Python : n'existe pas

A la place, il faut utiliser des elif :

`int jour;`
`printf("Entrez un jour : ");`
`scanf("%d", &jour);`
`printf("\n");`
`switch (jour) {`
`case 1 : printf("lundi\n"); break;`
`case 2 : printf("mardi\n"); break;`
`// et ainsi de suite`
`case 7 : printf("dimanche\n"); break;`
`default : printf("Erreur :\n");`
	`jour =int(input("Entrez un jour : "))`
	`if (jour==1):`
	`print("Lundi")`
	`elif (jour==2)`
	`print("Mardi")`
	`#et ainsi de suite`
	`else:`
	`print("Erreur : ")`

VII.COMMENTAIRES

1. *Pourquoi commenter un programme informatique*

Les commentaires sont utilisés dans les programmes informatiques pour plusieurs raisons :

1. **Documenter le code** : Les commentaires permettent d'expliquer le fonctionnement du code, les algorithmes utilisés, les décisions de conception prises, etc. Ils peuvent aider les développeurs à comprendre rapidement le code lorsqu'ils le relisent plus tard.
2. **Faciliter la maintenance** : Les commentaires peuvent aider les développeurs à comprendre et à corriger les erreurs dans le code plus facilement.
3. **Améliorer la lisibilité** : Les commentaires permettent de structurer le code en groupant les sections logiques et en fournissant une description concise des sections importantes.
4. **Partager avec d'autres développeurs** : Les commentaires peuvent aider les autres développeurs à comprendre le code et à travailler dessus plus efficacement.

En résumé, les commentaires sont un outil important pour les développeurs pour documenter le code, faciliter la maintenance et améliorer la lisibilité. Ils peuvent également être utiles pour partager le code avec d'autres développeurs et aider à transmettre la connaissance et les meilleures pratiques.

2. *Les commentaires*

Un code bien écrit doit être facilement compréhensif : en le lisant on doit comprendre ce que vous avez fait. Pour cela, il faut au moins utiliser des noms de variables qui indiquent leurs fonctions.Si cela n'est pas suffisant, les commentaires servent à préciser votre code.

a. Python

L'explication sur un ligne débute par « # » :

```
# Sur une ligne
```

Et sur plusieurs lignes, encadré par « """ »

```
""" Sur plusieurs
lignes   """
```

b. Langage C

Si le commentaire tient sur une seule ligne, on peut précéder l'explication par « // » :

```
// Sur une ligne
```

Pour plusieurs lignes, le commentaire doit être encadré par « /* » et « */ » à la fin :

```
/* Sur plusieurs
lignes   */
```

VIII. LES BIBLIOTHEQUES

1. *A quoi servent les bibliothèques*

Les directives `#include` dans le langage C ou les `include` de Python, servent à inclure des fichiers header (fichiers d'en-tête) dans un programme. Les fichiers header contiennent généralement des déclarations de constantes, de types, de variables et de fonctions qui peuvent être utilisées dans plusieurs programmes.

L'utilisation des fichiers header permet de :

1. **Éviter la duplication de code** : Au lieu de copier et de coller le même code dans plusieurs programmes, vous pouvez inclure le même fichier header dans plusieurs programmes.
2. **Améliorer la maintenance** : Si vous souhaitez faire une modification dans le code, vous n'avez qu'à le faire dans le fichier header, au lieu de le faire dans chaque programme qui utilise ce code.
3. **Faciliter la réutilisation du code** : Les fichiers header peuvent être utilisés pour créer des bibliothèques de code réutilisable.

En conclusion, les directives `#include` sont utiles pour inclure des déclarations de constantes, de types, de variables et de fonctions dans un programme, ce qui peut améliorer la lisibilité, la maintenance et la réutilisation du code.

2. Fonctionnement des include en langage C

Les `include` sont un mécanisme en C pour inclure des fichiers de en-tête dans un programme.
La syntaxe est : `#include <nom_du_fichier.h>`.
Les fichiers d'en-tête sont des fichiers qui contiennent des définitions de fonctions, de variables et de structures utilisées dans un programme.
L'utilisation des `include` permet de séparer le code en plusieurs fichiers pour une meilleure organisation et une maintenance plus facile.
Les `include` améliorent également la réutilisation de code en permettant de partager du code entre plusieurs programmes.

3. Les différences entre #include du langage C et import de Python

Les `#include` du langage C et les `import` de Python sont deux mécanismes différents pour inclure du code externe dans un programme.

Les différences sont :

- Le `#include` en C est utilisé pour inclure des fichiers d'en-tête qui contiennent des définitions de fonctions, de variables et de structures utilisées dans le programme. Les fichiers d'en-tête sont inclus lors de la **pré-compilation** du code source.
- L'`import` en Python est utilisé pour inclure des modules externes dans un programme. Les modules peuvent être utilisés pour fournir des fonctionnalités supplémentaires à un programme. Les modules sont **chargés à l'exécution** et peuvent être utilisés dans le code source.

En résumé, les `#include` en C sont utilisés pour inclure des définitions de fonctions, de variables et de structures, tandis que les `import` en Python sont utilisés pour inclure des modules externes.

4. Les bibliothèques (ou Library en anglais)

a. Python : import

L'équivalent à `#include` est **import** en langage Python.

Par exemple pour importer la fonction sqrt (square root = racine carrée) :

```
from math import sqrt
print(sqrt(16))
```

Il est possible d'en importer plusieurs à la fois :

```
from math import sqrt, pi, cos
print(cos(pi/2))
```

Et même toutes les fonctions du module math :

```
import math
```

```
print(math.sqrt(16)) # Affiche 4
```

Un autre exemple où en plus d'importer, on a défini un synonyme (PI en majuscule au lieu de pi)

```
from math import pi as PI
print(PI)
```

Et même renommer un module :

```
import numpy as np
import matplotlib.pyplot as plt
```

b. Langage C : include

La syntaxe est : `#include <nom_du_fichier.h>`.

Pour pouvoir utiliser `printf(…)` il faut mettre au début du code `#include <stdio.h>` pour que le compilateur sache que cette fonction est définie « ailleurs ».

```
#include <stdio.h>

int main ()
{
  printf("Hello World \n");
  return 0;
}
```

Toutes les fonctions nécessitent d'inclure des fichiers d'entête

IX. LES STRUCTURES DE CONTROLE

1. *A quoi sert la boucle pour*

La boucle `for` en langage C est utilisée pour **exécuter un bloc de code plusieurs** fois en fonction d'une condition de boucle spécifiée. La boucle `for` est souvent utilisée lorsque vous savez combien de fois vous voulez exécuter un bloc de code, ou lorsque vous voulez itérer sur les éléments d'un tableau.

Les trois éléments entre les parenthèses définissent la condition de boucle :

1. **Initialisation** : C'est l'expression qui définit la valeur de départ de la boucle. Elle est exécutée une seule fois au début de la boucle.
2. **Condition** : C'est l'expression qui définit la condition pour poursuivre ou arrêter la boucle. Si la condition est vraie, le bloc de code sera exécuté. Sinon, la boucle sera terminée.
3. **Incrémentation** : C'est l'expression qui est exécutée à la fin de chaque itération de la boucle et qui permet de mettre à jour la valeur de la condition de boucle.

2. *La boucle for pour répéter un certains nombre de fois*

Cette boucle est utilisée lorsqu'on connait le nombre d'itérations avant même de l'appeler c'est-à-dire :
La boucle pour est utilisée lorsque l'on connait le nombre de fois que la boucle va être parcourue

Par exemple au lieu d'écrire :

```
print('bonjour') # 1
print('bonjour') # 2
print('bonjour') # 3
print('bonjour') # 4
print('bonjour') # 5
print('bonjour') # 6
print('bonjour') # 7
print('bonjour') # 8
print('bonjour') # 9
print('bonjour') # 10
```

Il est possible d'utiliser la boucle *for* :

Langage C	Python

`int i ;`
`for (i = 0; i <= 9; i++)`
`  {`
`  printf("coucou\n") ;`
`  }`
	`for  i in range(0,10):`
	`  print('bonjour') #tabulation au début`

a. Python : toujours for

Comme pour le if, à la place des parenthèses du langage C, python utilise **l'indentation** qui est soit une tabulation, soit quatre espaces.

La fonction range()

La fonction *range* permet de créer une liste de nombres compris entre un nombre de départ (inclus) et un nombre de fin (exclus).

Range() a trois paramètres : début, fin et pas, exemple :

```
for i in range(10): # de 0 à 9 (10 non inclus)
for i in range(1,10): # de 1 à 9
for i in range(0,100,5): # de 0 à 95 par pas de 5
for i in range(0,-10,-3): # de 0 à -9 par pas de -3
```

Par exemple pour afficher 10 fois bonjour :

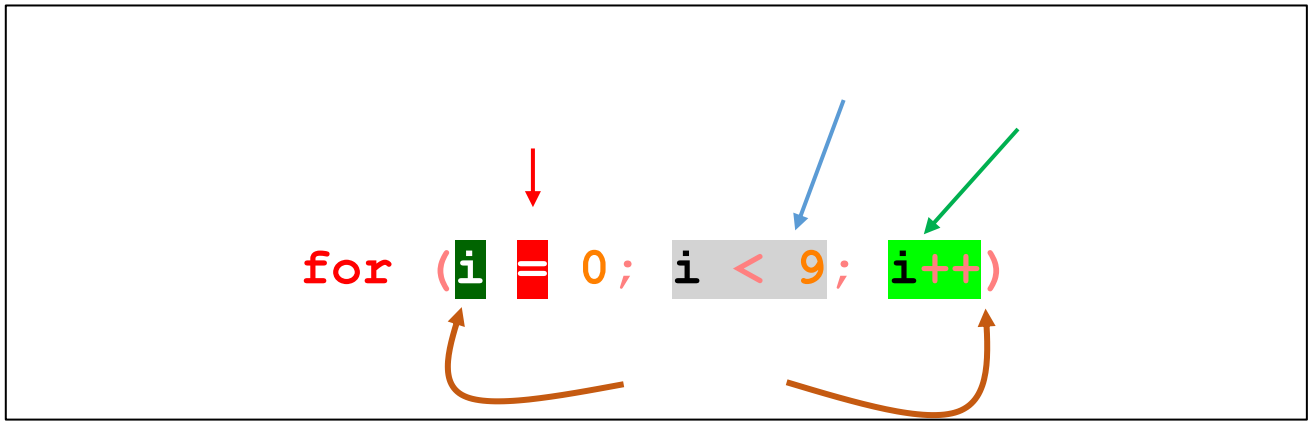
```
for  i in range(0,10):
    print('bonjour') # indentation au début de ligne
```

b. Langage C : for

Algo	Langage C

Pour i variant de 0 à 9
** afficher("coucou)**
FinPour
	`int i ;`
	`for (i = 0; i <= 9; i++)`
	`  {`
	`  printf("coucou\n") ;`
	`  }`

Explications :



Les trois éléments entre les parenthèses définissent la condition de boucle :

1. Initialisation : C'est l'expression qui définit la valeur de départ de la boucle. Elle est exécutée une seule fois au début de la boucle.
2. Condition : C'est l'expression qui définit la condition pour poursuivre ou arrêter la boucle. Si la condition est vraie, le bloc de code sera exécuté. Sinon, la boucle sera terminée.
3. Incrémentation : C'est l'expression qui est exécutée à la fin de chaque itération de la boucle et qui permet de mettre à jour la valeur de la condition de boucle.

Autres exemples

Exemple 1	Exemple 2

`char car;`
`for(car='A';car!='Z'+1;car++)`
`  printf("%c",car);`
	`char i;`
	`for (i = 0 ; i <= 100 ; i++)`
	`{`
	`  printf("i = %d\n", i); /*affiche les i de 0 à 100*/`
	`}`

Remarques

- Souvent on voit dans les codes `i++` c'est équivalent à `i=i+1` ou même à `i+=1`
- Généralement, en informatique, on commence à compter à partir de zéro :

`for (int i = 0; i < 9; i++)` est équivalent à `for (int i = 1; i <=10 ; i++)`

- `for ( ;;)` est une boucle infinie

c. Exemple simple

```c
for( i = 1; i <=10; i=i+1) {
    printf("i=%d\n",i);
    }
```

```python
for i in range(1,11):
    print("i=",i)
```

Boucles imbriquées :

```c
#include <stdio.h>

int main()
{
  int i, j;
  for (i = 0; i <= 100; i++)
  {
    for (j = 0; j <= 10; j++)
    {
      if (j % 2 == 0)
      {
        printf("Pair\n");
      }
      printf("%d\n", i * j);
    }
    printf("%d\n", i);

  }
  printf("Fin du programme\n");
  return 0;
}

                }
        printf("Fin du programme\n");
    return 0;
}
```

```python
for i in range(101):
  for j in range (10):
    print(j)
    if (j%2 == 0) :
      print('Pair') # dans le if
    print(i*j) # dans le 2 for
  print(i) # dans le 1er
print('Fin du programme') # à la fin
```

3. Pourquoi utiliser un while ?

Le `while` est un constructeur de boucle en langage C qui permet d'exécuter un bloc de code plusieurs fois en **fonction d'une condition spécifiée**. La boucle `while` est utilisée lorsque vous voulez répéter un bloc de code tant que la condition est vraie.

La syntaxe de la boucle `while` est la suivante :

```
while (condition) {
  // bloc de code à exécuter
}
```

Remarques

- Pas de point-virgule après la condition
- Présence de parenthèse pour isoler les instructions à exécuter

L'expression de condition définit la condition pour poursuivre ou arrêter la boucle. Si la condition est vraie, le bloc de code sera exécuté. Sinon, la boucle sera terminée.

En utilisant une boucle `while`, vous pouvez exécuter un bloc de code tant que la condition spécifiée est vraie. Cela est utile pour les tâches de boucle où le nombre de répétitions n'est pas connu à l'avance. Par exemple, vous pouvez utiliser une boucle `while` pour lire des entrées utilisateur jusqu'à ce qu'un certain critère soit rempli.

4. While pour répéter en respectant une condition

Cette boucle est utilisée quand on ne connait pas le nombre de fois que la boucle doit être itérée.

a. En python

C'est exactement la même syntaxe qu'en langage C. Il ne faut pas oublier les deux points « : » à la fin de la ligne et l'indentation (tabulation ou espace).

```
x=45
y=55
while x<50 and y <70:
    x=x+1
    y=y+1
    print(x,y)
```

b. Exemple en Langage C

Algo	Langage C

Somme<-0
Tant que somme<100
faire somme=2*somme+1
	`Somme=0 ;`
	`while (somme < 100)`
	`{`
	`somme = 2 * somme + 1;`
	`}`

Remarques

- Attention : La condition se trouvant au début de la boucle, il faut s'assurer qu'elle soit bien vérifiée. Dans l'exemple ci-dessus, Somme est mis à zéro
- Il n'y a pas de point-virgule à la fin de la ligne contenant le **`while`**

- **while (1) {…}** est une boucle infinie.

c. Exemples

Tant que la somme <100

```c
#include <stdio.h>
int main() {
  int somme = 0;
  while (somme < 100)
  {
    somme = 2 * somme + 1;
  }

  printf("somme=%d\n", somme);
// Affiche somme=127
  return 0;
}
```

```python
somme = 0
while somme<100:
    somme = 2*somme + 1
print('somme=',somme)
```

A la place d'une boucle for

Langage C	Python

`#include <stdio.h>`
`int main() {`
`for( i = 1; i <=10; i=i+1) {`
`printf("i =%d \n",i);`
`}`
`return 0;`
`}`
	`for i in range(1,11):`
	`print("i =",i)`

`#include <stdio.h>`
`int main() {`
`int i = 1;`
`while (i <= 10)`
`{`
`printf("i=%d\n", i);`
`i ++;`
`}`
`return 0;}`
	`i = 1`
	`while ( i <10 ):`
	`print("i =",i)`
	`i = i + 1`

5. *La boucle faire … tant que : uniquement en C*

La condition n'est vérifiée qu'à la fin de la boucle : celle-ci sera exécutée au moins une fois.

a. Exemple

Algo	Langage C

Réponse : caractère
répéter
Afficher ("Voulez-vous sortir (O/N) ?")
lire réponse
tant que réponse ≠ 'O'
	`char caractere ;`
	`do`
	`{`
	`printf("Voulez-vous sortir (O/N) ?");`
	`scanf("%c", &caractere);`
	`}`
	`while (caractere != 'O');`

b. Remarque

- Attention aux accolades et au point-virgule à la fin de while.

X. LES FONCTIONS

1. *Le concept de fonction*

Les fonctions déjà présentes jusqu'alors

Vous avez déjà utilisé en C/Python des fonctions : `printf(…), scanf(…), main()` ou `print(…), int(…), range(…)`
Elles sont reconnaissables par des parenthèses à la fin de leur nom. Elles ont toutes été conçues et mises au point par d'autres personnnes, puis réunies dans des librairies (C) ou modules (Python).

Besoin :

Dès que le programme commence à prendre de l'ampleur, il est nécessaire **d'écrire vos propres fonctions** : un programme de plus de cent lignes commencent à être illisible, difficile à corriger et des parties sont répétitives. De plus, il est nécessaire de **décomposer** votre travail, en sous-tâches, qui vont être codées en fonctions.

Une fonction c'est :

Une **suite instructions isolées** du reste du programme, qui possède un **nom**, et qui peut être **appelée** par ce nom à n'importe quel endroit du programme et autant de fois que l'on veut.

Points essentiels :

- Un programme écrit **sans fonction** devient **difficile** à comprendre dès qu'il dépasse une centaine de lignes
- Les fonctions permettent de **scinder** le programme principal en plusieurs parties
- Le **programme principal regroupe** les fonctions en décrivant les enchaînements
- Une fonction peut elle-même, être **découpée** en plusieurs autres fonctions.
- Et pour terminer, et peut-être le plus essentiel, les fonctions permettent le **partage de tâches** : dans un projet important où travaille plusieurs développeurs, chacun est responsable d'une ou de plusieurs fonctions, qui sont ensuite utilisées par les autres programmeurs et mise en commun dans le programme principal.

Dans cette partie, nous ne parlerons pas de modularité (découper le fichier principal en plusieurs fichiers), mais cette notion est indispensable pour les plus longs programmes.

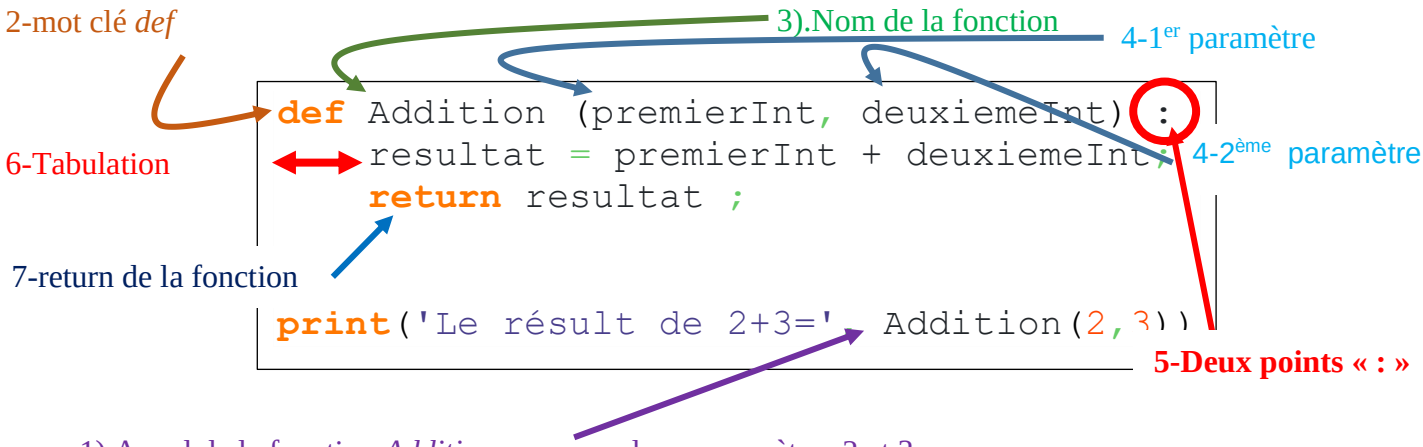
2. *Les questions qu'il faut se poser avant d'écrire une fonction*

1. **Quel est le but de la fonction ?**
   - Déterminez l'objectif principal de la fonction et ce qu'elle doit accomplir.
2. **Quels sont les arguments d'entrée de la fonction ?**
   - Identifiez les paramètres nécessaires pour accomplir la tâche de la fonction. Cela peut inclure des entiers, des flottants, des chaînes de caractères, des pointeurs, des structures ou d'autres types de données.
3. **Quelle est la valeur de retour de la fonction ?**
   - Déterminez le type de valeur que la fonction doit renvoyer pour indiquer le résultat de son exécution. Si la fonction n'a pas besoin de renvoyer de valeur, utilisez le type `void`.
4. **Comment gérer les erreurs ou les cas exceptionnels ?**
   - Pensez aux erreurs ou aux situations exceptionnelles qui peuvent survenir pendant l'exécution de la fonction et comment les gérer de manière appropriée.
5. **Comment nommer la fonction de manière claire et cohérente ?**
   - Choisissez un nom de fonction qui reflète clairement ce qu'elle fait et respecte les conventions de nommage du projet ou du langage.
6. **La fonction doit-elle être accessible depuis d'autres fichiers ou modules ?**
   - Si la fonction doit être accessible depuis d'autres fichiers, déclarez-la dans un fichier d'en-tête approprié et utilisez le mot-clé `extern` lors de la déclaration.
7. **La fonction doit-elle être réutilisable ou générique ?**
   - Si la fonction peut être utilisée dans divers scénarios, essayez de la rendre générique et réutilisable en évitant les dépendances spécifiques à une situation.
8. **Comment documenter la fonction ?**

- Assurez-vous de fournir des commentaires pour expliquer le but de la fonction, les paramètres d'entrée, les valeurs de retour et les erreurs ou situations exceptionnelles que la fonction peut gérer.

3. *Les fonctions en Python*

a. Exemple commenté




```
def Addition (premierInt, deuxiemeInt) :
    resultat = premierInt + deuxiemeInt;
    return resultat ;

print('Le résult de 2+3=', Addition(2,3))
```

1. Cette fonction *Addition* est appelée avec deux paramètres. *Addition* renvoie une valeur de retour.
2. Dans la déclaration de la fonction, le premier terme est **def**
3. Le second terme est le nom de la fonction
4. Ensuite les définitions des paramètres
5. **Deux points « : »**
6. **Une tabulation** (ou indentation)
7. Un renvoie de la valeur par *return*

Remarques :

- On retrouve les deux points et la tabulation des boucle *for* et *while*
- **Ne jamais mettre de print dans les fonctions, sauf nécessité, utilisez *return* pour renvoyer une valeur puis l'afficher.**
- L'instruction *return* renvoie une valeur et met fin à l'exécution de la fonction et redonne le contrôle à la fonction appelante.
- Par (mauvaises) habitudes de nombreux programmeurs utilisent des parenthèses pour encadrer la valeur de retour. *return* est une **instruction** pas une fonction

b. Fonction sans paramètre ne retournant pas de valeur

```
import random


def Affiche10Fois():
    '''
    Fonction sans paramètre
    ne retournant pas de valeur
    '''
    for i in range(10):
        print('coucou')
```

c. Fonction sans paramètre, retournant une valeur

```
def Aleatoire():
    '''
```

```
    Fonction sans paramètre,
    retournant une valeur
    '''
    return random.randint(0, 20)
```

d. Deux paramètres sans retour de valeur

```
def AfficherAddition(i, j):
    '''
    Fonction utilisant un ou plusieurs paramètres
    ne retournant pas de valeur
    '''
    print('addition entre',i,'et','j','est',i+j)
```

e. Plusieurs paramètres et retournant une valeur

```
def Discriminant(b, a, c):
    '''
    Fonction utilisant plusieurs paramètres
    retournant une valeur
    '''
    delta = b * b - 4 * a * c;
    return delta
```

f. Les fonctions sont appelées

```
Affiche10Fois()
nombre = Aleatoire()
print('un nombre aléatoire entre 0 et 20 : ', nombre);
AfficherAddition(2, 3)  # Affiche L'addition entre 2 et 3 est 5
nombre = Discriminant(6, 3, 2);
print('Le Discriminant est ', nombre); # Affiche le delta
```

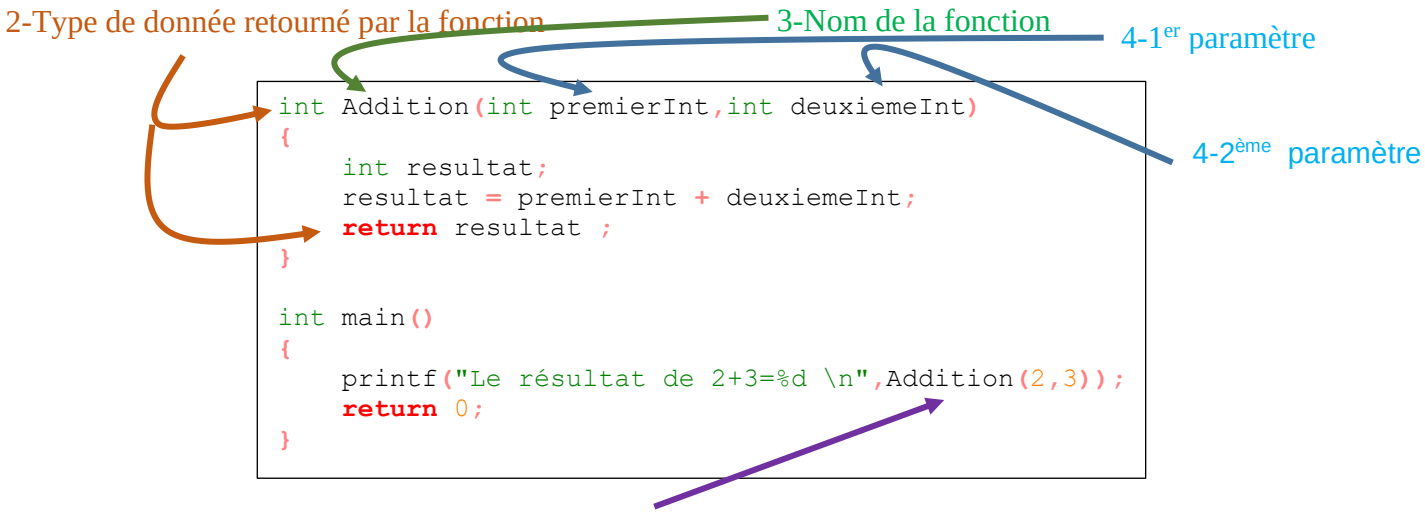
Remarques

- Quatre fonctions ont été écrites puis utilisées plus bas
- le programme principal devient alors court et lisible
- il est possible de « mimer » une fonction *main* comme étant le programme principal à l'aide de l'instruction

```
if __name__ == "__main__":
```

4. Les fonctions en Langage C

a. Exemple commenté



1. Cette fonction *Addition* est appelée dans le *main* avec deux paramètres déclarés comme étant des entiers au tout début de la définition de la fonction. *Addition* renvoie toujours dans le *main* une valeur de retour.
2. Dans la déclaration de la fonction, le premier terme indique le type de donnée qui va être retourné (ici un entier int)
3. Le second terme est le nom de la fonction
4. Ensuite les définitions des paramètres

Remarques :

- **Ne jamais mettre de printf dans les fonctions, sauf nécessité, utilisez *return* pour renvoyer une valeur puis l'afficher.**
- L'instruction *return* renvoie une valeur et met fin à l'exécution de la fonction et redonne le contrôle à la fonction appelante.
- Par (mauvaises) habitudes de nombreux programmeurs utilisent des parenthèses pour encadrer la valeur de retour. *return* est une **instruction** pas une fonction

b. Fonction sans paramètre ne retournant pas de valeur Exemples

```
#include <stdio.h>

void Affiche10Fois() //Fonction sans paramètre ne retournant pas de valeur
{
    int i;
    for( i = 0; i < 9; i++) printf("coucou\n") ;
}
```

c. Fonction sans paramètre, retournant une valeur

```
int Aleatoire() // Fonction sans paramètre, retournant une valeur
{
  srand(time(NULL)); // indispensable pour obtenir un nombre différent
                     //à chaque fois
  return (rand() % 20 );  // % 20 représente le reste de la division entière par 20
}
```

d. Deux paramètres sans retour de valeur

```
/* Fonction utilisant un ou plusieurs paramètres
    ne retournant pas de valeur */
void AfficherAddition(int i, int j)
{
    printf("L'addition entre %d et %d est %d \n",i,j,i+j);
}
```

e. Plusieurs paramètres et retournant une valeur

```
//Fonction utilisant un ou plusieurs paramètres, retournant une valeur
float Discriminant(float b, float a, float c)
{
    float delta = b*b - 4*a*c;
    return(delta);
}
```

f. Les fonctions sont regroupées dans le *main*

```
int main()
{
   int nombre;
   Affiche10Fois();
   nombre=Aleatoire();
   printf("un nombre aléatoire entre 0 et 20 : %d\n",nombre);
   AfficherAddition(2,3); // Affiche L'addition entre 2 et 3 est 5
   nombre = Discriminant(6,3,2);
   printf("Le Discriminant est %d \n",nombre); // Affiche L'addition entre 2 et 3 est 5
return 0;
}
```

Remarques

- Quatre fonctions ont été écrites puis utilisées dans *main*
- le programme principal devient alors court et lisible

XI. REPARTIR LES FONCTIONS DANS DES FICHIERS

Il est courant en programmation de séparer les fonctions d'un programme dans différents fichiers pour faciliter la maintenance et la réutilisation du code. Voici comment le faire en langage C :

a. Création de fichier d'entête

Créer un fichier .h (header) pour chaque fonction que vous souhaitez séparer. Ce fichier doit contenir le prototype de la fonction (c'est-à-dire la déclaration de la fonction) et toute autre information nécessaire, telle que les définitions de constantes ou de types. Par exemple :

```
// exemple.h

#ifndef EXEMPLE_H_INCLUDED
#define EXEMPLE_H_INCLUDED

void exemple();

#endif // EXEMPLE_H_INCLUDED
```

b. Création d'un fichier .c par type de fonction

Créer un fichier .c pour chaque fonction que vous souhaitez séparer. Ce fichier doit contenir la définition de la fonction (c'est-à-dire le code qui implémente la fonction) et toutes les autres fonctions et variables nécessaires à la fonction. Il doit également inclure le fichier .h correspondant pour que le compilateur puisse trouver le prototype de la fonction. Par exemple :

```
// exemple.c
```

```c
#include "exemple.h"

void exemple() {
    // code qui implémente la fonction exemple
}
```

c. Appel dans le programme principal

Dans votre programme principal, incluez le fichier .h pour chaque fonction que vous souhaitez utiliser, puis compilez tous les fichiers .c ensemble. Par exemple :

```c
// main.c

#include "exemple.h"

int main() {
    exemple();
    return 0;
}
```

d. Compilation

Compilez tous les fichiers .c ensemble pour créer l'exécutable final. Par exemple avec gcc :

gcc -o mon_programme main.c exemple.c

Cela crée un exécutable nommé "mon_programme" qui inclut les fonctions du fichier main.c et du fichier exemple.c. Les prototypes de chaque fonction sont définis dans leur fichier .h correspondant. Cela permet de séparer les fonctions dans des fichiers séparés et de faciliter la maintenance et la réutilisation du code.

2. Comparaison

Langage C

```c
#include <stdio.h>
#include <stdlib.h> // Pour rand
#include <time.h> // Pour time

void Affiche10Fois() //Fonction sans paramètre ne retournant pas de
valeur
{
    int i;
    for( i = 0; i < 9; i++) printf("coucou\n") ;
}

int Aleatoire() // Fonction sans paramètre, retournant une valeur
{
  srand(time(NULL)); // indispensable pour obtenir un nombre différent
                     //à chaque fois
  return (rand() % 20 );  // % 20 représente le reste de la division
entière par 20
}

/* Fonction utilisant un ou plusieurs paramètres
    ne retournant pas de valeur */
void AfficherAddition(int i, int j)
{
    printf("L'addition entre %d et %d est %d \n",i,j,i+j);
}

//Fonction utilisant un ou plusieurs paramètres, retournant une valeur
float Discriminant(float b, float a, float c)
{
    float delta = b*b - 4*a*c;
    return(delta);
}

int main()
{
   int nombre;
   Affiche10Fois();
   nombre=Aleatoire();
   printf("un nombre aléatoire entre 0 et 20 : %d\n",nombre);
   AfficherAddition(2,3); // Affiche L'addition entre 2 et 3 est 5
   nombre = Discriminant(6,3,2);
   printf("Le Discriminant est %d \n",nombre); // Affiche le delta

    return 0;
```

Python

```python
import random


def Affiche10Fois():
    '''
    Fonction sans paramètre
    ne retournant pas de valeur
    '''
    for i in range(10):
        print('coucou')


def Aleatoire():
    '''
    Fonction sans paramètre,
    retournant une valeur
    '''
    return random.randint(0, 20)


def AfficherAddition(i, j):
    '''
    Fonction utilisant un ou plusieurs paramètres
    ne retournant pas de valeur
    '''
    print('addition entre',i,'et','j','est',i+j)



def Discriminant(b, a, c):
    '''
    Fonction utilisant plusieurs paramètres
    retournant une valeur
    '''
    delta = b * b - 4 * a * c;
    return delta


Affiche10Fois()
nombre = Aleatoire()
print('un nombre aléatoire entre 0 et 20 : ', nombre);
AfficherAddition(2, 3)  # Affiche L'addition entre 2 et 3 est 5
nombre = Discriminant(6, 3, 2);
print('Le Discriminant est ', nombre); # Affiche le delta
```

CHAPITRE N°2 : TABLEAUX, CHAINES ET OPERATEURS

I. LES TABLEAUX

1. *Les tableaux à une dimension*

a. Exemples de déclarations

```
int tab1[5];                    // Déclaration d'un tableau de 5 entiers
flot tab2[3] = {1.1,2.2,3.3}; // Déclaration d'un tableau de 3 types char
char texte1[ ]="Bonjour";      // Déclaration d'un tableau de types char
                        // et initialisation de ce tableau avec les codes ASCII
                        // de la chaîne de caractères utilisée.
```

`int tab1[5];` Réserve l'emplacement pour 5 éléments de type int. Chaque élément est repéré par sa position dans le tableau nommé : indice.
En C, la première position porte le numéro 0. Donc ici les indices vont de 0 à 4 et le premier sera désigné par tab2[0] et le dernier tab2[4].

b. Initialisation

```
float tab2[3] = {1.2,2.3,3.4}; // toutes les valeurs
float tab2[3] = {1.2,2.3}; // les premières
float tab2[] = {1.2,2.3,3.4}; // le compilateur va créer trois emplacements
```

c. Remarque

- Aucun contrôle de débordement d'indice n'est mis en place ce qui peut, mais pas toujours, occasionner des erreurs.

d. Exemple d'utilisation moyenne de 20 notes

Les tableaux utilisent souvent la boucle *for* :

```
#include <stdio.h>
int main()
{ int i, som, nbm ;
  float moy ;
  int t[20] ;

  for (i = 0 ; i < 20 ; i++)
  {
```

```
    printf ("donnez la note numéro %d : ", i + 1) ;
    scanf ("%d", &t[i]) ;
  }
  for (i = 0, som = 0 ; i < 20 ; i++) som += t[i] ;
  moy = som / 20 ;
  printf ("\n\n moyenne de la classe : %f\n", moy) ;

  for (i = 0, nbm = 0 ; i < 20 ; i++ )
    if (t[i] > moy) nbm++ ;

  printf ("%d élèves ont plus de cette moyenne", nbm) ;
  return 0;

}
```

2. *Les tableaux à plusieurs dimensions*

a. Déclarations – exemples

En langage C, il est possible de définir des tableaux à plusieurs dimensions comme étant des tableaux de tableaux.

Exemples :

```
1. float x[10];// Une dimension
2. float point_2D[10][2];//10 points 2D avec ses coodonnées x,y
3. float point_2D[10][3];//10 points 3D avec ses coodonnées x,y,y
4. float notes[8][6];//6 notes pour 8 élèves
5.
6.  float notesElevesTrimestre [24][10][3];
7.  // les notes au maxi 10 pour les 24 élèves et pour les 3 trimestres
```

1. Déclaration d'un tableau à une dimension contenant, par exemple, les abscisses de 10 points-> **10 éléments**
2. Tableau de 10 éléments qui possède chacun 2 autres éléments. Dans l'exemple ce sont les enregistrements des coordonnées x et y de 10 points **->10*2=20 éléments**
3. Toujours un tableau à deux dimensions mais cette fois avec 3 éléments par point (x,y et z) **10*3=30 éléments**
4. Tableau de notes obtenues par des élèves à des devoirs (au maxi 6 notes et 8 élèves). **8*6=48 éléments**

6. Les notes des élèves de la classe par rapport aux devoirs pour les trois trimestres d'une année scolaire : **24*10*3=240*3=720 éléments**

b. Accès

Par convention, les éléments déclarés dans un tableau *tab[x][y]* à 2 dimensions sont repérés par :

- *tab[lig][col]*
- où *lig* représente la ligne
- et *col* la colonne :

Le numéro de la ligne, *lig*, est compris entre 0 et x-1
Le numéro de la colonne, *col*, est compris entre 0 et y-1

	Colonne 0	Colonne1	Colonne2
Ligne 0	x[0][0]	x[0][1]	x[0][2]
Ligne 1	x[1][0]	x[1][1]	x[1][2]
Ligne 2	x[2][0]	x[2][1]	x[2][2]

c. Représentation graphique

Deux dimensions

Soit : `float notes[24][10];//10 notes pour 24 élèves`

devoirs

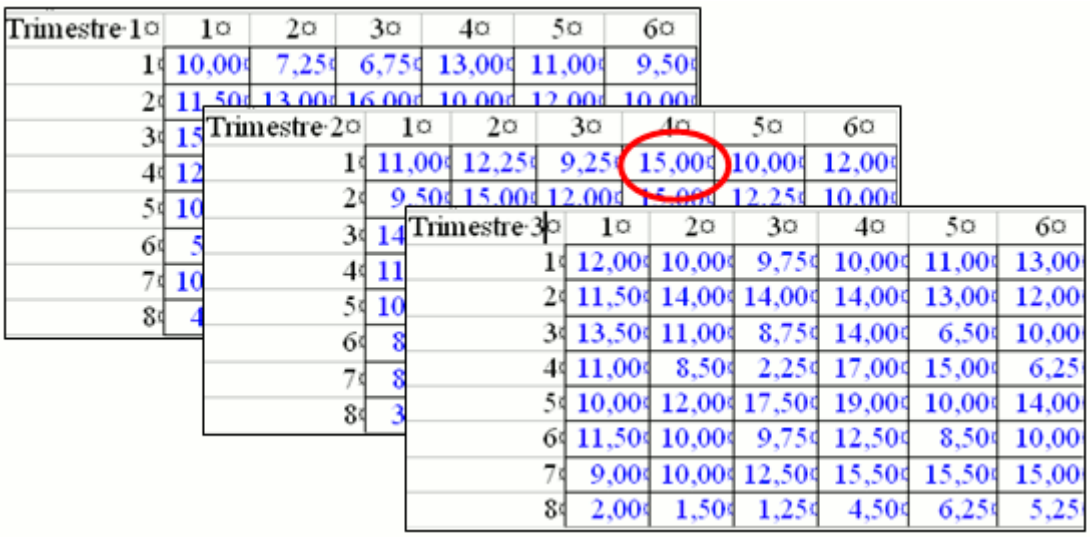
élèves

10,00	7,25	6,75	13,00	11,00	9,50
11,50	13,00	16,00	10,00	12,00	10,00
15,00	9,00	8,75	14,00	6,50	10,00
12,50	9,50	7,25	15,00	13,00	7,25
10,00	11,00	15,50	18,00	14,00	10,00
5,50	6,25	9,75	7,50	6,50	9,50
10,00	11,00	18,75	17,50	12,50	11,00
4,50	6,50	3,25	6,50	9,25	8,75

L'accès à la cinquième note du second élève est `notes[1][4]` donne la valeur 12.

Trois dimensions

`float notesElevesTrimestre [24][10][3];` représente les notes des élèves de la classe par rapport aux devoirs pour les trois trimestres d'une année scolaire.



`notesElevesTrimestre[0][3][1]` représente la 4ème note du premier élève au trimestre 2 = **15,00**

d. Initialisation

a) Deux dimensions

La déclaration et l'initialisation peuvent se faire par :

```
int x[3][4] = {0,1,2,3,4,5,6,7,8,9,10,11}
```

Mais il est préférable de la faire ainsi :

```
int x[3][4] = {{0,1,2,3}, {4,5,6,7}, {8,9,10,11}};
```

Ou pour encore plus de visibilité :

```
 int x[3][4] = {
      {0,1,2,3},
      {4,5,6,7},
      {8,9,10,11}
      };
```

b) Trois dimensions

De même, il est possible d'écrire :

```
int x[2][3][4] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10,
                  11, 12, 13, 14, 15, 16, 17, 18, 19,
                  20, 21, 22, 23
                 };
```

Mais il est préférable de la faire ainsi :

```
int x[2][3][4] =
    {
        { {0,1,2,3}, {4,5,6,7}, {8,9,10,11} },
        { {12,13,14,15}, {16,17,18,19}, {20,21,22,23} }
    };
```

a. Exemples

```
    int x[2][3][2] =
    {
        { {0,1}, {2,3}, {4,5} },
        { {6,7}, {8,9}, {10,11} }
    };

    // affiche les valeurs de tous les éléments
    for (int i = 0; i <= 1; ++i)
    {
        for (int j = 0; j <=2; ++j)
        {
            for (int k = 0; k <=1; ++k)
            {
                printf("L'element a x[%d][%d][%d] = %d\n",i,j,k,x[i][j][k]);
            }
        }
    }
```

```
L'element a x[0][0][0] = 0
L'element a x[0][0][1] = 1
L'element a x[0][1][0] = 2
L'element a x[0][1][1] = 3
L'element a x[0][2][0] = 4
L'element a x[0][2][1] = 5
L'element a x[1][0][0] = 6
L'element a x[1][0][1] = 7
L'element a x[1][1][0] = 8
L'element a x[1][1][1] = 9
L'element a x[1][2][0] = 10
L'element a x[1][2][1] = 11
```

II. LES CHAINES DE CARACTERES

1. Déclaration

Une chaîne de caractères (string en anglais) est une suite de caractères stockée dans un tableau de char et terminée par le caractère '\0'.
Le '\0' marque la fin de la chaîne quelle que soit la taille du tableau mais le nombre total de caractères dans la chaîne ne peut pas dépasser la taille du tableau.
Par exemple :

```
1.   char Str1[15];
2.   char Str2[8] = {'a', 'r', 'd', 'u', 'i', 'n', 'o'};
3.   char Str3[8] = {'a', 'r', 'd', 'u', 'i', 'n', 'o', '\0'};
4.   char Str4[ ] = "arduino";
5.   char Str5[8] = "arduino";
6.   char Str6[15] = "arduino";
```

1. Str1 : déclare un tableau de char sans initialisation.
2. Str2 : déclare un tableau de char (avec un élément supplémentaire) et le compilateur va rajouter le caractère null obligatoire
3. Str3 : ajout explicite du caractère null
4. Str4 : initialise avec une chaîne de caractère entre guillemets (double), le compilateur va dimensionner le tableau pour l'ajuster à la taille de la chaîne et va ajouter le caractère null
5. Str5 : initialise le tableau avec une taille explicite et une chaîne de caractères.
6. Str6 : initialise le tableau et laisse un espace supplémentaire pour une chaîne de caractères plus importante.

2. Ajout du caractère nul

```
char texte1[ ]="Bonjour";     // Déclaration d'un tableau de types char
                          // et initialisation de ce tableau avec les codes
```

- L'initialisation du tableau texte1 remplit la mémoire de la manière suivante :

'B'	'o'	'n'	'j'	'o'	'u'	'r'	'\0'

'B', 'o', 'n', 'j', 'o', 'u', 'r' représentant les codes ASCII respectifs des caractères du tableau. Et '\0' va être ajouté automatiquement par le compilateur.
Le caractère '\0' étant le caractère de code ASCII 0, indiquant la fin de la chaîne de caractère.
Ce tableau comprend donc non pas 7 caractères, mais 8 caractères.

3. Plusieurs dimensions = liste de chaînes

Il est possible de représenter une liste de chaînes de caractères à l'aide d'un tableau à deux dimensions, par exemple comme ceci :

```
char noms[][16]={"Alice", "Bob", "Samia", "Zheng-You"};
printf(noms[1]);// Affiche Bob
```

4. Fonctions de manipulations

Des fonctions, se trouvant dans <string.h> permettent de manipuler ces chaînes de caractères :

Nom de la fonction	Utilité	Exemple
strlen	Renvoie la taille	`Printf("La taille est %d\n",strlen(nom))`
strcpy	Copie	`strcpy(nom2,nom);// Destination en premier, strncpy copie n caractères`
strncpy	Copie les n premiers caractères	`strncpy(nom2,nom,5);// Destination en premier, strncpy copie n caractères`
strcat	Ajoute	`strcat(nom,nom2); // Ajoute nom2 à la fin de nom`
strncat	Ajoute n caractères	`strncpy(nom,nom2,5); // Ajoute les 5 premiers caractères de nom2 à la fin de nom`
strcmp	Compare deux chaines	`if (strcmp(nom,"Nestor")==0) // strcmp = 0-> égaux`
strncmp	Compare les n premiers caractère de deux chaines	`if (strncmp(nom,"Nestor",2)==0) // strcmp = 0-> égaux, strncmp compare les deux premiers caractères`
atoi	Converti une chaîne en int (entier)	`int entier= atoi("12");// Entier vaut 12`
sprintf	Stocke la chaîne formatée dans une autre chaîne	`int entier=12; sprintf(formatage,"un entier = %d",entier); // formatage[]="un entier = 12"`

5. Exemple d'utilisation : une chaîne=un tableau

```
#include <stdio.h>
#include <string.h>

int main()
{
    char texte[10];
    int indice,longueur;

    printf("texte ? ");
    scanf("%s", texte);
    longueur = strlen(texte); // longueur de la chaîne sans le '\0'
    printf("La taille du texte est de %d\n",longueur);
```

```
texte ? azer
La taille du texte est de 4
indice=0 caractere=a
indice=1 caractere=z
indice=2 caractere=e
indice=3 caractere=r
```

```
    for (indice = 0; indice <= longueur - 1; indice++)
        printf("indice=%d caractere=%c\n", indice,texte[indice]);

}
```

Dans l'exemple, *texte* est composé de cinq caractères : 'a', 'z', 'e', 'r' et '\0'. Mais *strlen(texte)* renvoie 4 car il y a 4 caractères non nuls.
L'indice commençant à 0 : texte[0]='a' jusqu'à texte[3]='r' -> l'indice va donc de 0 à strlen -1

6. Utilisation de tableau de char à la place des int

Pour rappel :

char	1	-128 (2^7) à 127 (2^7-1)	Représente un seul caractère (code ASCII) et peut être utilisé pour un « petit » entier compris entre -128 et 127.
unsigned char	1	0 à 255 (2^8-1)	Représente un seul caractère (code ASCII) ou un entier compris entre 0 et 255.
int	2 (µp 16 bits) 4 (32 bits)	-32768 (-2^{15}) à 32767 (2^{15}-1)	Entier positif ou négatif.
unsigned int	idem	0 à 65535 (2^{16}-1)	Uniquement les entiers positifs appelés « non signés »

Utilisation de char pour économiser de la mémoire :
Au lieu d'utiliser des entiers qui vont prendre chacun 4 octets il est possible d'utiliser un char à la place qui ne va prendre qu'un seul octet (à condition que le nombre ne soit pas supérieur à 127/255)

Si la variable est comprise entre -128 et + 127, pour économiser de la mémoire on va préférer utiliser un type char :

```
int tab2[3] = {-1,2,3}; // Déclaration d'un tableau de 3 entiers = 12 octets
char tab2[3] = {-1,2,3};      // Déclaration d'un tableau de 3 types char = 3 octets
```

Si la variable n'est uniquement positif :

```
unsigned char tab2[3] = {1,2,3};
```

III. LES OPERATEURS

1. Définition

Fonctions	O	Description	Exemples
Identificateurs	()	Appel de fonction	
	[]	Indice de tableau	tableau[3]=5;
opérateurs unaires	!	Négation logique (NOT)	b=!a; (si a>0 => b=0, si a=0 =>b=1)
	~	Complément binaire bit à bit	b=~a
	-	Moins unaire	b=-a;
	+	Plus unaire	b=+a;
	++	Préincrément ou postincrément	b=a++; (b=a puis a=a+1)
	--	Prédécrément ou postdécrément	b=a--; (b=a puis a=a-1)
	&	Adresse de	b=&a; (b égale l'adresse de a)
	*	Indirection (adressage indexé)	b=*a; (b=contenu de l'adresse de a)
opérateurs binaires	*	Multiplication	c=a*b;

	/	Division	c=a/b;
	+	Plus binaire	c=a+b;
	-	Moins binaire	c=a-b;
	<<	Décalage à gauche	c=a<<b; (a est décalé b fois à gauche)
	>>	Décalage à droite	c=a>>b; (a est décalé b fois à droite)
	&	ET entre bits	c= a & b; (ET logique bit à bit)
	^	OU exclusif entre bits	c= a ^b;
	|	OU entre bits	c= a|b;
Tests	<	Strictement inférieur	if a < b
	<=	Inférieur ou égal	if a >= b
	>	Strictement supérieur	if a > b
	>=	Supérieur ou égal	if a >= b
	==	Egal	if a ==b (si a est égale à b)
	!=	Différent	if a != b
	&&	ET logique	if ((a=5) && (b=2))
	||	OU logique	if ((a=5) ||(b=2))
	?:	Condition	z=(a>b)?a:b (Si a>b a z=a sinon z=b)
Affectation	=	Affectation simple	a=b; (a prend la valeur de b)
Auto-affectations	*=	Affectation produit	a*=2 (a=a*2)
	/=	Affectation quotient	a/=2 (a= a/2)
	%=	Affectation reste	a%=2 (a= le reste de a/2)
	+=	Affectation somme	a+=2 (a=a+2)
	-=	Affectation différence	a-=2 (a=a-2)
	&=	Affectation ET entre bits	a&=5 (a=a&5)
	^=	Affectation OU EX entre bits	a^=5 (a=a^5)
	|=	Affectation OU entre bits	a|==5 (a=a|=5)
	<<=	Affectation décalage gauche	a<<=5 (a=a<<5)
	>>=	Affectation décalage droite	a>>=5 (a=a>>5)

2. *Exemple*

```
#include <stdio.h>
char a, b, c;
int i1, i2, i3, i4; long l1, l2; float f1, f2, f3; double d1;

main()
{
i1 = i2 = i3 = 11;      /* voila des exemples d'affectations multiples */
a = b = c = 40;
f1 = f2 = f3 = 12.987;

/* Opérations arithmétiques de base */
i3 = -i2;   /* donne i3 = -11 */
i3 = i1 + i2;     /* donne i3 = 22 */
i3 = i1 - i2;     /* donne i3 = 0 */
i3 = i1 * i2;     /* donne i3 = 121 */
i3 = i1 / i2;     /* donne i3 = 1 */
i3 = 15+(i1 = 4*i2);          /* donne i1= 44 puis i3 = 59 */
i4 = i3 / i2;     /* donne i4 = 5 */
i4 = i3 % i2;     /* % = modulo donc i4 = 8 */
i4 = i3 >> 2;     /* i4 = i3 apr,s 2 décaléges … droite (*4) */
i4 = i3 << 3;     /* i4 = i3 apr,s 3 décaléges … gauche (/8) */

/* Opérations arithmétiques de niveau 2 : automodifications */
a = a + 1;  /* incrémentation de a */
a += 1;     /* idem */
a++;  /* idem APRES utilisation de a */
++a;  /* idem AVANT utilisation de a */
a = a - 1 ; /* décrémentation de a */
a -= 1;     /* idem */
```

```
a = 40;
b = a--;     /* b = 40 puis a = 39 */
b = --a;     /* a = 38 puis b = 38 */
a += 10;     /* a+10, a = 48 */
b -= 5;      /* b-5, b = 33 */
f1 *= 1.5;   /* f1 * 1.5, f1 = 8.658 */
f2 /= 2.2;   /* f2 / 2.2, f2 = 5.9032 */
a <<= 1;     /* a décalé 1 fois à gauche a = 24 */
b >>= 2;     /* b décalé 2 fois à droite b = 132 */

/* Opérateurs Logiques de niveau 1 */
b = 15; c = 12;
a = b & c;  /* a = b ET c donc a = 10 */
a = b | c;  /* a = b OU c donc a = 17 */
a = b ^ c;  /* a = b OU EXCLUSIF c donc a = 7 */
a = ~b;     /* a = NOT b donc a = $EF */

/* Opérations de niveau 3 : Affectation conditionnelle */
/* syntaxe générale : (Test) ? action_si_oui : action_si_non; */
i1 = (i2 > 3) ? 2 : 10;  /* Cette instruction doit se lire:
si i2 est supérieur à 3 alors i1 devient 1 sinon, i1 devient 10 */
c = (a > b) ? a : b;    /* c est le maxi d'entre a et b */
c = (a > b) ? b : a;    /* c est le mini d'entre a et b */

c = (a >= 0) ? a : -a;  /*  c est la valeur absolue de a */

/* Opérateurs divers */
a = sizeof (f1);  /* sizeof retourne la taille en octet de */
b = sizeof (l1);  /* l'expression paramètre */
f2 = int(15) / int(4);  /* conversion de type : CAST. En absence des */
f2 = int (15/4);  /* commandes de cast, f2 vaudrait 3.75, ici 3 */

/* Comparaisons logiques de base */
if (i1 == i2) i= -13;   /* puisque i1 = i2, i3 deviendra -13 */
if (i1 > i3) a = 'A';         /* donne 'A' (65) … la variable a */
if (!(i1 > i3)) a = 'B';      /* puisque i1 >i3, a ne change pas */
if (b <= c) f1 = 0.0;   /* puisque b = c alors f1 devient 0 */
if (f1 != f2) f3 = i3/2;      /* comme f1 <> f2, f3 devient 6.5 */

/* Comparaisons logiques de niveau 2 */
/* bas,es sur le fait qu'une valeur vraie est diff,rente de 0 et qu'une valeur fausse est
nulle */
if (i1 = (f1 != f2)) i3 = 111;      /* comme f1 <> f2, le résultat du test
est vrai, le résultat logique du test vrai est affecté à i1 et i3 change */
if (i1) i3 = 222;       /* i1 est vrai donc i3 change */
if (i1 != 0) i3 = 333;  /* i1 vrai donc non nul donc i3=333 */
if (i1 = i2) i3 = 444;  /* i2 recoit i1, cette valeur non nulle
donc vraie  i3 = 222 */

/* Comparaisons logiques niveau 3 : Tests multiples */
i1 = i2 = i3 = 77;
if ((i1 == i2) && (i1 == 77)) i3 = 33;    /* && = ET logique: i3 change */
if ((i1 > i2) || (i3 > 12)) i3 = 22;      /* || = OU logique: i3 change */
if (i1 && i2 && i3) i3 = 11;  /* tous non nuls donc i3 change */
if ((i1=1)&&(i2=2)&&(i3=3)) i3 = 44;      /* Attention, affectations des
variables donc i3 change */
if ((i1==2)&&(i2=3)) i3 = 55; /* i1 <> 2 donc i3 ne change pas */
}
```

CHAPITRE N°3 : LECTURE ET ECRITURE DANS UN FICHIER

La lecture de fichiers est une fonctionnalité essentielle dans de nombreux programmes informatiques. Elle permet à un programme de récupérer des données stockées dans un fichier externe pour les traiter ou les afficher à l'utilisateur. En langage C, la lecture de fichiers se fait grâce à des fonctions de la bibliothèque standard telles que fopen, fscanf ou fgets. La maîtrise de ces fonctions est donc cruciale pour tout programmeur souhaitant manipuler des fichiers en langage C.

I. LECTURE DE FICHIERS

A MODIFIER AVEC PRINCIPALEMENT FPRINTF ET FSCANF

1. Ouverture d'un fichier

L'ouverture d'un fichier est la première étape pour pouvoir le lire en langage C. Pour ouvrir un fichier, on utilise la fonction `fopen()` qui prend en paramètres le nom du fichier et le mode d'ouverture "r" pour read.

```
FILE *fichier = fopen("fichier.txt", "r");
```

2. Lecture de caractères

Pour lire un fichier caractère par caractère, on utilise la fonction `fgetc()`. Cette fonction lit un caractère du fichier à chaque appel et renvoie sa valeur ASCII. Pour s'assurer que le fichier est ouvert, on vérifie que la valeur renvoyée n'est pas égale à EOF (End Of File).

3. Lecture de lignes

Pour lire un fichier ligne par ligne, on utilise la fonction `fgets()`. Cette fonction lit une ligne du fichier à chaque appel et renvoie un pointeur sur la chaîne de caractères correspondante. Si la fin du fichier est atteinte, la valeur NULL est renvoyée.
Une boucle est souvent utilisée :

```
while (fgets(ligne, MAX_SIZE, fichier) != NULL) {
    printf("%s", ligne);
}
```

4. Lecture de blocs de données

Pour lire un fichier par blocs de données, on utilise la fonction `fread()`. Cette fonction lit un bloc de données de taille donnée à chaque appel et renvoie le nombre d'éléments lus. Si la fin du fichier est atteinte, la valeur renvoyée est inférieure au nombre d'éléments demandés.

5. Lecture formatée : utilisation de fscanf

Il est également possible d'utiliser `fscanf` qui s'utilise comme la fonction `scanf` avec les entrées d'utilisateurs. Cette méthode est moins facile à utiliser que les autres fonctions `fgets` ou `fgetc`

6. Fermeture d'un fichier

La fermeture d'un fichier est importante pour libérer les ressources utilisées par le système. Pour fermer un fichier, on utilise la fonction `fclose()` qui prend en paramètre le pointeur vers le fichier à ferme

```
    fclose(fichier);
```

7. Exemples de code

a. En utilisant fgetc

Code	Affichage

```c
#include <stdio.h>

int main() {

    // Ouverture du fichier en mode lecture
    FILE *fichier = fopen("mon_fichier.txt", "r");

    // Vérification si l'ouverture a réussi
    if (fichier == NULL) {
        printf("Erreur lors de l'ouverture du fichier.\n");
        return 1;
    }

    // Lecture caractère par caractère jusqu'à la fin du fichier
    char c;
    while ((c = fgetc(fichier)) != EOF) {
        printf("%c", c); // Affichage du caractère lu
    }

    // Fermeture du fichier
    fclose(fichier);

    return 0;
}
```

Affichage :

Première ligne
Deuxième ligne
Troisième ligne
Dernière ligne

b. En utilisant fgets

Code	Affichage

```c
// Inclusion de la bibliothèque standard d'entrées/sorties
#include <stdio.h>

// Définition d'une constante pour la taille maximale de la ligne
#define MAX_SIZE 100

// Fonction principale
int main() {
  // Ouverture du fichier en mode lecture seule
FILE *fichier = fopen("mon_fichier.txt", "r");

// Vérification de la réussite de l'ouverture du fichier
if (fichier == NULL) {
    printf("Impossible d'ouvrir le fichier.");
    return 1; // On quitte le programme avec un code d'erreur
}

// Déclaration d'un tableau de caractères pour stocker chaque ligne
char ligne[MAX_SIZE];

// Boucle de lecture des lignes du fichier, tant qu'on n'a pas
atteint la fin
while (fgets(ligne, MAX_SIZE, fichier) != NULL) {
```

Affichage :

Première ligne
Deuxième ligne
Troisième ligne
Dernière ligne

```c
    // Affichage de la ligne lue
    printf("%s", ligne);
}

// Fermeture du fichier
fclose(fichier);

// Fin du programme sans erreur
return 0;}
```

c. Lecture d'une chaine et d'un entier avec fscanf

```c
#include <stdio.h>
#include <string.h>
#define MAX_PARKING 3
#define MAX_NOM 10


int main() {
    char parking[MAX_PARKING][MAX_NOM], nom_lu[MAX_NOM] ;
      int places[MAX_PARKING], place_lue;
  int index=0;


    FILE *fichier_parking = fopen("parkings.txt", "r");
    if (fichier_parking == NULL) {
        printf("Impossible d'ouvrir le fichier.");
        return 1;
    }

    while (!feof(fichier_parking) && index < MAX_PARKING) { // Boucle de lecture des
scores
      fscanf(fichier_parking,"%s %d",nom_lu,&place_lue );
      //  printf("%s %d", nom_lu, place_lue);
      strcpy(parking[index],nom_lu);
      places[index] = place_lue;
      index ++;

    }

  printf("Données lues : \n");

  for (int i=0;i<MAX_PARKING;i++) printf("%s %d\n",parking[i],places[i]);

    fclose(fichier_parking);
    printf("Nombre de places disponibles ecrit dans parking.txt.\n");
    return 0;
}
```

II. ECRITURE DANS DES FICHIERS

L'écriture dans un fichier consiste à écrire des données dans un fichier stocké sur le disque dur. Voici les étapes pour écrire dans un fichier en langage C :

1. Ouverture d'un fichier en écriture

La première étape consiste à ouvrir le fichier en mode écriture.
Pour cela, on utilise la fonction `fopen()` avec le mode "`w`" mis pour Write. Si le fichier n'existe pas, il sera créé. Si le fichier existe déjà, son contenu sera écrasé.

2. Écriture de caractères

Pour écrire un caractère dans un fichier, on utilise la fonction `fputc()`. Cette fonction prend en argument le caractère à écrire ainsi que le pointeur vers le fichier.

3. Écriture de chaînes de caractères

Pour écrire une chaîne de caractères dans un fichier, on utilise la fonction `fputs()` . Cette fonction prend en argument la chaîne de caractères à écrire ainsi que le pointeur vers le fichier.

4. Écriture de données binaires

Pour écrire des données binaires dans un fichier, on utilise la fonction `fwrite()`. Cette fonction prend en argument un pointeur vers les données à écrire, la taille des données en octets, le nombre d'éléments à écrire et le pointeur vers le fichier.

Attention : il est nécessaire d'ouvrir le fichier en mode binaire

5. Écriture formatée : utilisation de fprintf

Il est également possible d'utiliser `fprintf` qui s'utilise comme la fonction printf pour l'affichage à l'écran. Cette méthode est moins facile à utiliser que les autres fonctions `fputs` ou `fputc`.

6. Fermeture d'un fichier

Il est important de fermer le fichier après avoir fini d'écrire dedans. Pour cela, on utilise la fonction `fclose()` qui prend en argument le pointeur vers le fichier.

7. Exemples

a. Avec uniquement fputs

Code

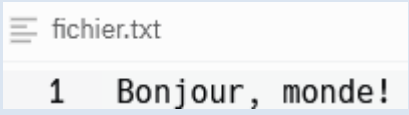
```c
#include <stdio.h>

int main() {
    // Ouverture du fichier en mode "écriture"
    FILE *fichier = fopen("fichier.txt", "w");
    if (fichier == NULL) { // Vérification si l'ouverture a réussi
        printf("Impossible d'ouvrir le fichier.");
        return 1;
    }

    char chaine[] = "Bonjour, monde!"; // Définition d'une chaîne de caractères à écrire dans le fichier
    fputs(chaine, fichier); // Écriture de la chaîne dans le fichier

    fclose(fichier); // Fermeture du fichier
    return 0;
}
```

fichier .txt



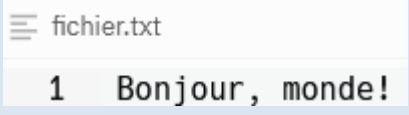
b. Avec fputs et putc

Code

```c
#include <stdio.h>

int main() {
    FILE *fichier; // déclaration d'un pointeur de type FILE
    fichier = fopen("fichier.txt", "w"); // ouverture du fichier en mode écriture
    if (fichier == NULL) { // vérification si l'ouverture a réussi ou non
```

fichier .txt



```
        printf("Impossible d'ouvrir le fichier."); //
message d'erreur si l'ouverture a échoué
        return 1; // arrêt du programme avec un code
d'erreur
    }

    char ch = 'A'; // déclaration d'une variable caractère
    fprintf(fichier, "%c", ch); // écriture du caractère
dans le fichier

    char chaine[] = "Bonjour le monde !"; // déclaration
d'une chaîne de caractères
    fprintf(fichier, "%s", chaine); // écriture de la chaîne
dans le fichier

    int nombre = 42; // déclaration d'une variable entière
    fwrite(&nombre, sizeof(int), 1, fichier); // écriture
des données binaires dans le fichier

    fclose(fichier); // fermeture du fichier
    return 0; // arrêt du programme avec un code de succès
}
```

c. fputs fputc et fwrite

Code

```
#include <stdio.h>

int main() {
    FILE *fichier; // déclaration d'un pointeur de type FILE
    fichier = fopen("fichier.txt", "w"); // ouverture du fichier en mode écriture
    if (fichier == NULL) { // vérification si l'ouverture a réussi ou non
        printf("Impossible d'ouvrir le fichier."); // message d'erreur si l'ouverture a échoué
        return 1; // arrêt du programme avec un code d'erreur
    }

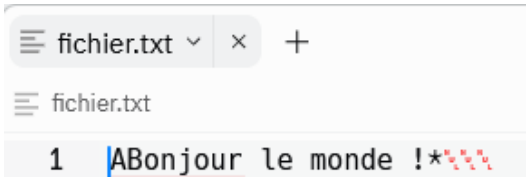
    char ch = 'A'; // déclaration d'une variable caractère
    fprintf(fichier, "%c", ch); // écriture du caractère dans le fichier

    char chaine[] = "Bonjour le monde !"; // déclaration d'une chaîne de caractères
    fprintf(fichier, "%s", chaine); // écriture de la chaîne dans le fichier

    int nombre = 42; // déclaration d'une variable entière
    fwrite(&nombre, sizeof(int), 1, fichier); // écriture des données binaires dans le
fichier

    fclose(fichier); // fermeture du fichier
    return 0; // arrêt du programme avec un code de succès
}
```

Le fichier est



d. Même code avec uniquement printf

```
#include <stdio.h>

int main() {
    FILE *fichier; // déclaration d'un pointeur de type FILE
    fichier = fopen("fichier.txt", "w"); // ouverture du fichier en mode écriture
    if (fichier == NULL) { // vérification si l'ouverture a réussi ou non
```

```
        printf("Impossible d'ouvrir le fichier."); // message d'erreur si l'ouverture a échoué
        return 1; // arrêt du programme avec un code d'erreur
    }

    char ch = 'A'; // déclaration d'une variable caractère
    fprintf(fichier, "%c", ch); // écriture du caractère dans le fichier

    char chaine[] = "Bonjour le monde !"; // déclaration d'une chaîne de caractères
    fprintf(fichier, "%s", chaine); // écriture de la chaîne dans le fichier

    int nombre = 42; // déclaration d'une variable entière
    fprintf(fichier, "%d", nombre); // écriture de l'entier dans le fichier

    fclose(fichier); // fermeture du fichier
    return 0; // arrêt du programme avec un code de succès
}
```

e. Ecriture d'une chaine et d'un entire en même temps à l'aide de fprintf

```
#include <stdio.h>
#include <string.h>
#define MAX_PARKING 3
#define MAX_NOM 10


int main() {
    char parking[MAX_PARKING][MAX_NOM] = {"Lille", "Roubaix", "Tourcoing"};
      int places[MAX_PARKING] = {521,605,755};


    FILE *fichier_parking = fopen("parkings.txt", "w");
    if (fichier_parking == NULL) {
        printf("Impossible d'ouvrir le fichier.");
        return 1;
    }

    for (int i = 0; i < MAX_PARKING; i++) {
        fprintf(fichier_parking, "%s  %d\n", parking[i], places[i]);
    }

    fclose(fichier_parking);
    printf("Nombre de places disponibles ecrit dans parking.txt.\n");
    return 0;
}
```

8. Conclusion

Vous avez donc le choix d'utiliser soit de toujours utiliser `fprintf` ou alors `fputs`, `fputc` ou `fwrite` suivant vos besoins.

III. MANIPULATION DE FICHIERS

La manipulation de fichiers consiste à effectuer des opérations autres que la simple lecture et l'écriture de fichiers. Cette section présente les opérations les plus courantes telles que le renommage, la suppression, la vérification de l'existence et le déplacement des fichiers.

1. Pour lire ou écrire dans un fichier quand faut-il utiliser "wr" ou "rb" ?

En langage C, "r" et "w" sont des modes d'ouverture de fichier pour la lecture et l'écriture respectivement. "b" est utilisé pour indiquer que le fichier doit être ouvert en mode binaire plutôt qu'en mode texte. Les combinaisons "rb" et "wb" sont donc utilisées pour ouvrir un fichier en mode binaire pour la lecture ou l'écriture.

En général, vous devez utiliser "rb" lorsque **vous lisez ou écrivez des données binaires**, telles que des images, des fichiers audio ou vidéo, ou des fichiers de base de données. Vous devez utiliser "r" ou "w" lorsque vous travaillez avec des fichiers texte, tels que des fichiers de configuration, des fichiers de journalisation, ou des fichiers de texte brut.

Il est important de noter que si vous utilisez "rb" pour lire ou écrire dans un fichier texte, les caractères de fin de ligne ne seront pas traités correctement. De même, si vous utilisez "r" ou "w" pour lire ou écrire dans un fichier binaire, vous risquez d'avoir des problèmes avec la représentation des données binaires.

Exemple de lecture d'un fichier image et de l'affichage de son contenu en hexadécimal :

```c
#include <stdio.h>

int main() {
    FILE *fichier;
  unsigned char unOctet;
    fichier = fopen("Web.png", "rb");
    if (fichier == NULL) {
        printf("Impossible d'ouvrir le fichier.");
        return 1;
    }


    while ((unOctet = fgetc(fichier)) !=0xff) {
        printf("%x ", unOctet);
    }

    fclose(fichier);
    return 0;
}
```

```
Console × Shell × +
> make -s
> ./main
89 50 4e 47 d a 1a a 0 0 0 d 49 48 44 52 0 0 0 65 0 0 0 65 8 6
0 0 0 54 7c 2d cf 0 0 0 1 73 52 47 42 0 ae ce 1c e9 0 0 0 4 67
41 4d 41 0 0 b1 8f b fc 61 5 0 0 0 9 70 48 59 73 0 0 e c3 0 0 e
 c3 1 c7 6f a8 64 0 0 31 4a 49 44 41 54 78 5e ed 9d 7 74 55 e7
b1 b6 71 12 3 c1 6 a1 de b 42 20 21 a1 de 7b ef bd 57 d4 b ea d
 75 d4 bb 28 a2 37 49 f4 de c1 74 4c b1 29 c6 d8 18 db b8 b7 24
 4e 4f d6 bd c9 bd b9 e9 81 f7 9f f9 f6 3e 92 70 48 b9 b9 3f b6
 93 e8 5b 6b d6 3e 3a 1c c1 39 fb 39 ef bc 33 b3 b d3 a6 d6 d4
9a 5a 53 6b 6a 4d ad a9 35 b5 a6 d6 d4 9a 5a 53 6b 6a 4d ad a9
35 b5 fe e1 f5 8c 61 c9 e 2d e7 be 61 e7 c0 4d 9d a1 e1 a3 6d 5
 21 5b ba fb 42 36 35 9f 8f 1a ad fc 28 6c 73 cd cf 82 36 55 >
```

Et l'affichage est :

2. Renommer un fichier :

Pour renommer un fichier, on utilise la fonction "rename()" fournie par la bibliothèque standard "stdio.h". Cette fonction prend deux arguments : le nom actuel du fichier et le nouveau nom que l'on souhaite donner au fichier.
Exemple :

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    char ancienNom[] = "mon_fichier.txt"; // Déclaration d'une variable contenant l'ancien nom du fichier
    char nouveauNom[] = "mon_fichier_renomme.txt"; // Déclaration d'une variable contenant le nouveau nom du fichier

    if (rename(ancienNom, nouveauNom) != 0) { // Tentative de renommage du fichier en utilisant la fonction rename()
        printf("Le renommage a échoué.\n"); // Message d'erreur si le renommage a échoué
        return 1; // Arrêt du programme avec un code d'erreur
    }

    printf("Le fichier a été renommé avec succès.\n"); // Message de succès si le renommage a réussi
    return 0; // Arrêt du programme avec un code de succès
}
```

3. Supprimer un fichier :

Pour supprimer un fichier, on utilise la fonction "remove()" fournie par la bibliothèque standard "stdio.h". Cette fonction prend en argument le nom du fichier à supprimer.
Exemple :

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    char nom_fichier[] = "mon_fichier.txt"; // nom du fichier à supprimer
    int etat_suppression; // variable pour stocker l'état de la suppression du fichier

    etat_suppression = remove(nom_fichier); // suppression du fichier et récupération de l'état de la suppression

    if (etat_suppression == 0) { // si l'état de la suppression est égal à 0, cela signifie que la suppression a été effectuée avec succès
        printf("Le fichier a été supprimé avec succès.");
    } else { // sinon, la suppression a échoué
        printf("La suppression du fichier a échoué.");
    }

    return 0;
}
```

4. Tester l'existence d'un fichier :

Pour tester si un fichier existe ou non, on utilise la fonction "access()" fournie par la bibliothèque standard "unistd.h". Cette fonction prend en argument le nom du fichier ainsi que le mode d'accès que l'on souhaite tester. Si le fichier existe et que le mode d'accès est autorisé, la fonction retourne 0. Sinon, elle retourne une valeur différente de 0.

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    char nom_fichier[] = "mon_fichier.txt"; // Nom du fichier à tester
    FILE *fichier; // Pointeur vers le fichier
```

```c
    // Ouverture du fichier en mode lecture
    fichier = fopen(nom_fichier, "r");

    // Vérification si l'ouverture a réussi
    if (fichier == NULL) {
        printf("Le fichier %s n'existe pas.\n", nom_fichier);
    } else {
        printf("Le fichier %s existe.\n", nom_fichier);
        fclose(fichier); // Fermeture du fichier
    }

    return 0;
}
```

5. Déplacer un fichier :

Pour déplacer un fichier, on utilise la fonction "rename()" fournie par la bibliothèque standard "stdio.h". Cette fonction prend deux arguments : le nom actuel du fichier et le nouveau chemin où l'on souhaite déplacer le fichier.

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Déclaration des variables
    char chemin_source[100]; // Chemin du fichier source
    char chemin_destination[100]; // Chemin du fichier de destination

    // Saisie des chemins des fichiers
    printf("Entrez le chemin du fichier source : ");
    scanf("%s", chemin_source);

    printf("Entrez le chemin du fichier de destination : ");
    scanf("%s", chemin_destination);

    // Tentative de déplacement du fichier
    int resultat = rename(chemin_source, chemin_destination);

    // Vérification si le déplacement a réussi ou non
    if (resultat == 0) {
        printf("Le fichier a été déplacé avec succès.\n");
    } else {
        printf("Impossible de déplacer le fichier.\n");
    }

    return 0;
}
```

6. Copier un fichier

Il est également possible de copier un fichier avant de le déplacer en utilisant la fonction "fopen()" pour ouvrir le fichier source et la fonction "fwrite()" pour écrire son contenu dans le fichier de destination. Ensuite, on peut supprimer le fichier source en utilisant la fonction "remove()".
La bibliothèque standard C fournit également une fonction appelée copyfile() dans la bibliothèque <copyfile.h>

Exemples

```c
#include <stdio.h>

#define TAILLE_BUFFER 1024 // Définir une taille de tampon pour la lecture/écriture par blocs


int main() {
```

```c
    FILE *source, *destination; // Déclarer des pointeurs de type FILE pour les fichiers source et destination
    char buffer[TAILLE_BUFFER]; // Déclarer un tampon pour la lecture/écriture par blocs
  int NbOctetsLu; // Déclarer une variable pour stocker le nombre d'octets lus

    // Ouvrir le fichier source en mode lecture binaire
    source = fopen("source.txt", "rb");
    if (source == NULL) { // Vérifier si l'ouverture a réussi ou non
        printf("Impossible d'ouvrir le fichier source.");
        return 1; // Retourner un code d'erreur et arrêter le programme
    }

    // Ouvrir le fichier destination en mode écriture binaire
    destination = fopen("destination.txt", "wb");
    if (destination == NULL) { // Vérifier si l'ouverture a réussi ou non
        printf("Impossible d'ouvrir le fichier destination.");
        return 1; // Retourner un code d'erreur et arrêter le programme
    }

    // Lire le contenu du fichier source par blocs et écrire dans le fichier destination

    do {
        NbOctetsLu = fread(buffer, 1, TAILLE_BUFFER, source); // Lire un bloc de données depuis le fichier source
        fwrite(buffer, 1, NbOctetsLu, destination);
        // Écrire le bloc de données lu dans le fichier destination
    } while (NbOctetsLu == TAILLE_BUFFER); // Continuer tant qu'il reste des données à lire

    // Fermer les fichiers source et destination
    fclose(source);
    fclose(destination);

    printf("Le fichier a été copié avec succès.");

    return 0; // Retourner un code de succès
}
```

CHAPITRE N°4 : ALLER PLUS LOIN AVEC LES TYPES ET LES VARIABLES

Dans ce chapitre, nous allons aller plus loin avec les variables et leurs types. Nous allons voir la différence entre variables locales et globales, créer de propres types de variables et pour finir transposer les concepts d'énumérations et de structures en langage C.

IV. LA CREATION DE NOUVEAUX TYPES DE VARIABLES

Précédemment nous avons vu les types de données couramment utilisés en C : char, int, float etc.. Et bien maintenant nous allons créer nos propres types.

1. Présentation

Le langage C vous permet de créer vos propres types de données à l'aide de *typedef* généralement déclaré en début de programme.

Les objectifs de `typedef` est de simplifier la déclaration de types et faciliter la lecture du code
La syntaxe est la suivant :

typedef type existant nouveau type.

2. Exemple

Par exemple

```
typedef unsigned char OCTET ;
```

Cette instruction définit un nouveau type de variables `OCTET` qui est un synonyme de `unsigned char`.
Par convention, les types définis par l'utilisateur sont écrit en majuscule, ce n'est pas obligatoire, mais cela permet de les distinguer lors de la lecture du code.

```
#include <stdio.h>
#include <stdlib.h>

typedef unsigned char OCTET;

int main() {
    OCTET octet = 255;
    printf("La valeur de l'octet est : %d\n", octet);
    return 0;
}
```

3. Rappel :

`unsigned char` est un type de données entières non signées en langage C.
Il est représenté sur 1 octet (8 bits) et peut contenir des valeurs comprises entre 0 et 255.
La différence entre `char` et `unsigned char` est que char est signé, c'est-à-dire qu'il peut représenter des valeurs positives et négatives entre -128 et +127, tandis que `unsigned char` ne peut stocker et des valeurs positives (de 0 à 255 donc).

Ainsi un `unsigned char` vous avez deux fois plus de valeurs qu'avec un `char`

V. LE TYPE ENUMERATION

1. Objectif et exemple

Quelques fois il est utile d'avoir une variable qui peut prendre uniquement quelques valeurs. L'exemple le plus classique est le booléen : soit vrai ou faux.

En langage C il est possible de créer de type de variable avec enum comme ceci :

```
#include <stdio.h>

enum boolean {
  false,
  true
};
```

```c
int main (void)
{
  enum boolean unBooleen;
  unBooleen = false;
  if (unBooleen == true)
  {
    printf ("VRAI (TRUE)\n");
  }
  else
  {
    printf ("FAUX (FALSE)\n");
  }
  return 0;
}
```

2. *Avec typedef*

L'utilisation d'un `typedef` avec `enum` permet de définir un nouveau type de donnée plus clair et plus expressif pour le programmeur. Cela facilite la lecture et la compréhension du code, en donnant un nom clair et évocateur à l'ensemble de valeurs possibles pour cette énumération.

`enum` est souvent utilisé *typedef :*

Sans typedef	Avec typedef

`#include <stdio.h>`
 	
`enum boolean {`
`  false,`
`  true`
`};`
 	
`int main (void)`
`{`
`  enum boolean unBooleen;`
`  unBooleen = false;`
`  if (unBooleen == true)`
`  {`
`  printf ("VRAI (TRUE)\n");`
`  }`
`  else`
`  {`
`  printf ("FAUX (FALSE)\n");`
`  }`
`  return 0;`
`}`
	`#include <stdio.h>`
	
	`typedef enum {`
	`  false,`
	`  true`
	`} BOOLEAN;`
	
	`int main (void)`
	`{`
	`  BOOLEAN unBooleen;`
	`  unBooleen = 3;`
	`  if (unBooleen == true)`
	`  {`
	`  printf ("VRAI (TRUE)\n");`
	`  }`
	`else`
	`  {`
	`  printf ("FAUX (FALSE)\n");`
	`}`
	`return 0;`
	`}`

VI. LES STRUCTURES

1. *Introduction*

a. Définition d'une structure

Les structures sont un élément fondamental du langage C. Elles permettent de créer des types de données personnalisés qui regroupent plusieurs variables de types différents sous un même nom.

Les structures sont souvent utilisées pour représenter des objets du monde réel ou des entités abstraites dans un programme informatique.

b. Pourquoi utiliser des structures ?

Elles sont également très utiles pour organiser et manipuler des données complexes de manière efficace. Les structures en C sont similaires aux classes en langages orientés objet, mais avec des fonctionnalités plus limitées.

c. Syntaxe d'une structure

Une structure est définie à l'aide du mot-clé `struct` suivi du nom de la structure et de ses champs, qui sont déclarés entre accolades. La syntaxe générale est la suivante :

```
struct NomDeLaStructure {
    typeDeChamp1 champ1;
    typeDeChamp2 champ2;
    ...
};
```

où `NomDeLaStructure` est le nom de la structure et `champ1`, `champ2`, ... sont les noms des champs de la structure, chacun étant associé à un type de données spécifique. Les champs peuvent être de n'importe quel type de données, y compris d'autres structures, de pointeurs, d'entiers, de caractères, de flottants, etc.

Une fois la structure définie, il est possible de déclarer des variables de ce type en utilisant le nom de la structure comme suit :

```
struct NomDeLaStructure variable;
```

Les champs de la structure peuvent être accessibles en utilisant l'opérateur point `.` pour accéder à un champ spécifique, comme ceci :

```
variable.champ1 = valeur1;
variable.champ2 = valeur2;
```

Les structures peuvent également être passées en tant que paramètres de fonction, ce qui permet de les manipuler et de les modifier de manière efficace dans une fonction.

2. *Déclaration et utilisation des structures*

a. Déclaration d'une structure

En langage C, une structure est un type de données personnalisé qui permet de regrouper plusieurs variables de types différents sous un même nom. Pour déclarer une structure, il faut utiliser le mot-clé "struct" suivi du nom de la structure et des variables qu'elle contient. Par exemple

```
struct Etudiant {
    char nom[50];
    char prenom[50];
    int age;
    float note;
};
```

Dans cet exemple, on définit une structure appelée "Etudiant" qui contient quatre variables : un tableau de caractères "nom", un tableau de caractères "prenom", un entier "age" et un flottant "note".
On peut ensuite créer des variables de type "Etudiant" pour stocker les informations d'un étudiant particulier.

b. Initialisation d'une structure

Pour initialiser une structure, il suffit de créer une variable de type "`struct`" et d'affecter les valeurs souhaitées à chacune de ses variables.
Par exemple :

```
struct Etudiant etud1;
strcpy(etud1.nom, "Dupont");
strcpy(etud1.prenom, "Jean");
etud1.age = 20;
```

```
etud1.note = 15.5;
```

Dans cet exemple, on crée une variable de type "Etudiant" appelée "etud1" et on lui affecte les valeurs "Dupont" pour le nom, "Jean" pour le prénom, 20 pour l'âge et 15.5 pour la note. On peut ensuite accéder aux variables de la structure en utilisant la notation pointée, par exemple "`etud1.nom`" pour accéder au nom de l'étudiant.

c. Accès aux membres d'une structure

Pour accéder aux membres d'une structure, on utilise l'opérateur point '.' suivi du nom du membre.

Exemple :

```
struct Personne {
    char nom[50];
    int age;
};

struct Personne p;
p.age = 25; // Accès au membre age de la structure Personne via l'opérateur point
```

3. *Questions à se poser*

Lorsque vous écrivez une structure en langage C, il est important de se poser certaines questions pour garantir que la structure correspond à vos besoins et qu'elle est bien conçue. Voici quelques questions à prendre en compte :

a. Quel est le but de la structure ?

Réfléchissez à l'objectif principal de la structure et à la manière dont elle sera utilisée dans votre programme.

b. Quels types de données la structure doit-elle contenir ?

Identifiez les types de données nécessaires pour représenter les informations que vous souhaitez stocker dans la structure. Cela peut inclure des types de base tels que `int`, `float`, `double`, `char`, ou d'autres structures ou tableaux.

c. Quelles sont les relations entre les données de la structure ?

Évaluez comment les différentes données de la structure sont liées entre elles. Cela peut vous aider à déterminer si elles doivent être regroupées dans une seule structure ou divisées en plusieurs structures.

d. La structure doit-elle être compatible avec d'autres structures ou fonctions du programme ?

Assurez-vous que la structure est compatible avec les autres parties de votre programme et qu'elle peut être facilement utilisée avec les fonctions existantes ou futures.

e. Dois-je créer des fonctions spécifiques pour manipuler cette structure ?

Pensez aux opérations que vous devrez effectuer sur la structure et déterminez si des fonctions spécifiques sont nécessaires pour manipuler les données. Ces fonctions peuvent inclure des fonctions d'initialisation, de copie, d'affichage, de comparaison, de modification ou de destruction.

4. Structures imbriquées et pointeurs vers des structure

a. Déclaration de structures imbriquées

Les structures imbriquées sont des structures qui contiennent d'autres structures. Elles permettent d'organiser les données de manière hiérarchique et de créer des types de données complexes. Pour déclarer une structure imbriquée, il suffit de définir une structure à l'intérieur d'une autre structure.

Exemple :

```
struct Date {
   int jour;
   int mois;
   int annee;
};

struct Personne {
   char nom[50];
   int age;
   struct Date date_naissance;
};
```

b. Accès aux membres de structures imbriquées

Pour accéder aux membres d'une structure imbriquée, il faut utiliser l'opérateur point '.'. L'opérateur point permet d'accéder aux membres d'une structure, qu'elle soit imbriquée ou non. Il suffit d'indiquer le nom de la structure, suivi d'un point et du nom du membre.

Dans cet exemple, la structure Personne contient un champ qui est également une structure. Et bien tout simplement, nous allons accéder au premier champ par un point et au second par un autre point :

```
struct Personne p1;
p1.age = 25;
strcpy(p1.nom, "Dupont");
p1.date_naissance.jour = 12;
p1.date_naissance.mois = 5;
p1.date_naissance.annee = 1995;
```

5. Tableaux de structures

a. Déclaration d'un tableau de structures

En C, il est possible de déclarer des tableaux de structures, qui sont des collections d'éléments de même type. Pour déclarer un tableau de structures, comme avec des tableaux d'entiers ou de caractères, il suffit de préciser le nom de la structure, suivi du nom du tableau et de sa taille entre crochets.
Par exemple, pour déclarer un tableau de structures "Personne" de taille 10 :

```
typedef struct {
    char nom[50];
    int age;
} Personne;

Personne tableau_personnes[10];
```

b. Accès aux éléments d'un tableau de structures

Pour accéder aux éléments d'un tableau de structures, on peut utiliser l'opérateur "[]" comme pour les tableaux classiques, suivi de l'indice de l'élément souhaité.
Par exemple, pour accéder à la 3ème personne du tableau "tableau_personnes" :

```
Personne personne_3 = tableau_personnes[2];
```

c. Exemples d'utilisation de tableaux de structures

Les tableaux de structures sont utiles pour stocker des collections d'objets de même type, tels que des employés, des clients, des produits, etc. Ils peuvent être utilisés pour trier et filtrer des données, ou pour stocker des informations dans des fichiers.
Par exemple, pour stocker une liste d'employés dans un tableau de structures "Employe" :

```
typedef struct {
    char nom[50];
    char prenom[50];
    int age;
    float salaire;
} Employe;

Employe liste_employes[100];

// Remplissage du tableau avec des données
liste_employes[0] = {"DUPONT", "Jean", 30, 2500.00};
liste_employes[1] = {"DURAND", "Marie", 25, 2000.00};
liste_employes[2] = {"MARTIN", "Pierre", 35, 3000.00};
...
```

CHAPITRE N°5 : LES POINTEURS – LES ADRESSES

I. INTRODUCTION

En langage C, un pointeur est une variable qui contient l'adresse mémoire d'une autre variable.
Il permet de manipuler directement la mémoire de l'ordinateur, en accédant à des emplacements spécifiques de la mémoire.
Les pointeurs sont largement utilisés en C pour manipuler des tableaux, des chaînes de caractères, des structures, et d'autres types de données complexes.
Ils sont également très utiles pour l'allocation dynamique de mémoire, c'est-à-dire pour réserver de la mémoire à l'exécution du programme, ce qui permet de créer des structures de données plus flexibles et plus efficaces.

Dans ce contexte, les pointeurs sont donc un élément clé du langage C, et leur maîtrise est indispensable pour tout programmeur C.

II. LES VARIABLES LOCALES ET GLOBALES

1. Les variables et la mémoire

En langage C, chaque variable est stockée en mémoire dans une zone réservée. La taille de cette zone dépend du type de la variable. Par exemple, un entier de type `int` est stocké sur 4 octets, tandis qu'un caractère de type `char` est stocké sur 1 octet.

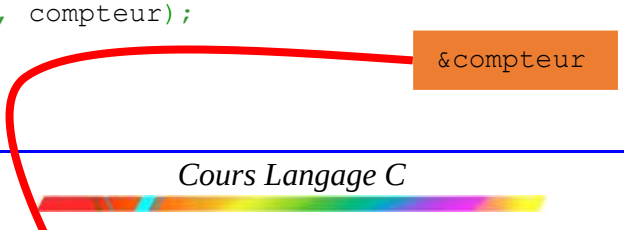
Chaque variable est également associée à une adresse mémoire, qui représente l'emplacement où elle est stockée en mémoire. Cette adresse peut être obtenue à l'aide de l'opérateur `&`.

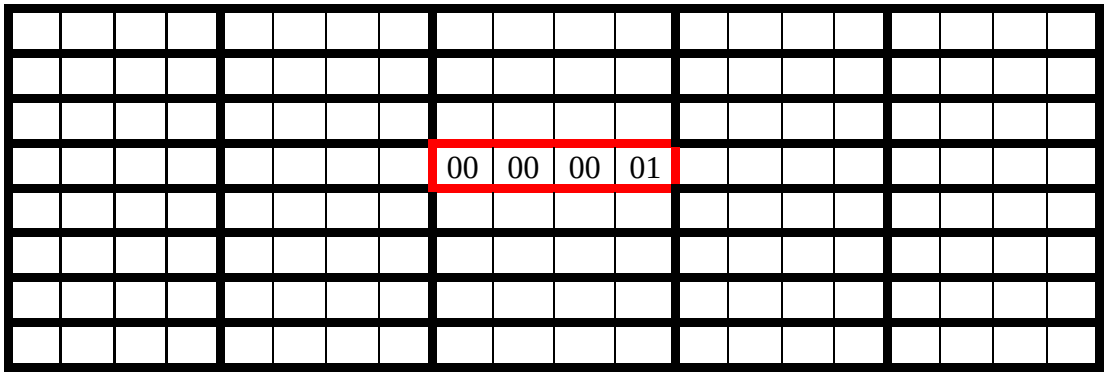
Lorsqu'une variable est déclarée, une zone de mémoire est allouée pour cette variable. Cette zone de mémoire peut être initialisée avec une valeur par défaut, ou bien être laissée non initialisée.

Lorsqu'une variable est utilisée dans le code, la valeur stockée dans la zone de mémoire associée à cette variable est lue ou modifiée en fonction des instructions du programme.

```c
#include <stdio.h>

int main() {
  int compteur = 1;
  printf("Compteur = %d", compteur);
compteur    return 0;
}
```





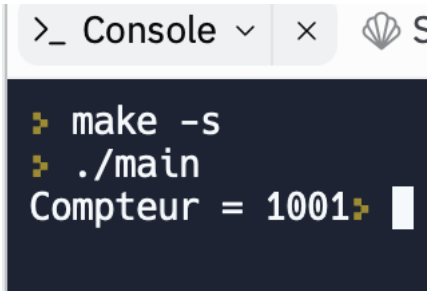
Mémoire

2. *Les variables locales*

Jusqu'à présent, nous avons utilisé, sans le savoir, des variables locales :

```c
#include <stdio.h>

int main() {
  int annee = 2023;
  printf("L'année est %d", annee);
  return 0;
}
```



Une variable locale est, comme son nom l'indique, déclarer localement. Le plus souvent c'est dans le *main()* mais pas uniquement : quel est l'affichage de ce programme ?

```c
#include <stdio.h>

int main() {
  int annee = 2023;
  {
    int annee = 2024;
  }
  printf("L'année est %d", annee);
  return 0;
}
```

C'est toujours 2023 qui est affiché.

Attention :
D'une manière générale, même si le compilateur fait la distinction entre les deux variables portant le même nom, pour des questions de lisibilité mais aussi et surtout pour éviter la confusion, éviter de nommer des variables avec le même nom.

Il en est de même pour les paramètres des fonctions. Par exemple même si `annee` représente la même chose pour vous, pour le compilateur ce sont deux variables différentes qui sont stockées dans deux espaces mémoires différents.

```c
#include <stdio.h>
```

```c
int lAnneeSuivantestBissextile(int annee) {
  annee++;
    if ((annee % 4 == 0 && annee % 100 != 0) || annee % 400 == 0) {
        return 1; // l'année est bissextile
    }
    else {
        return 0; // l'année n'est pas bissextile
    }
}

int main() {
    int annee = 2023;
    if (lAnneeSuivantestBissextile(annee)) {
        printf("L'année suivant %d est bissextile.\n", annee);
    } else {
        printf("%d n'est pas une année bissextile.\n", annee);
    }
    return 0;
}
```

Dans l'exemple, les deux variables `annee` ne partagent pas le même espace mémoire, ainsi lorqu'on incrément la variable `annee` c'est celle de la fonction qui est modifiée pas celle du `main()` :

```c
1  #include <stdio.h>
2
3  int lAnneeSuivantestBissextile(int annee) {
4    annee++;
5      if ((annee % 4 == 0 && annee % 100 != 0) ||
6          return 1; // l'année est bissextile
7      }
8      else {
9          return 0; // l'année n'est pas bissextil
10     }
11 }
12
13 int main() {
14     int annee = 2023;
15     if (lAnneeSuivantestBissextile(annee)) {
16         printf("L'année suivant %d est bissextil
17     } else {
18         printf("%d n'est pas une année bissextil
19     }
20     return 0;
```

Stack

main

annee int 2023

lAnneeSuivantestBissextile

annee int 2024

C/C++ details: none [default view]

3. Les variables globales

Le langage C autorise l'utilisation de variables globales : une fois déclarées en dehors du `main()` elles sont utilisables dans tout le programme, dans toutes les fonctions.

a. Exemple de variable globale

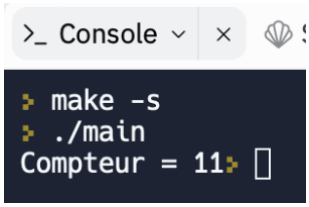
Soit le programme :

```c
#include <stdio.h>
int compteur=10;

int main()
{
```

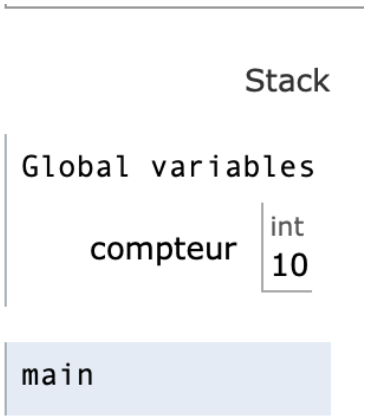
```
  compteur ++;
  printf("Compteur = %d",compteur);
    return 0;
}
```

L'affichage est le suivant :

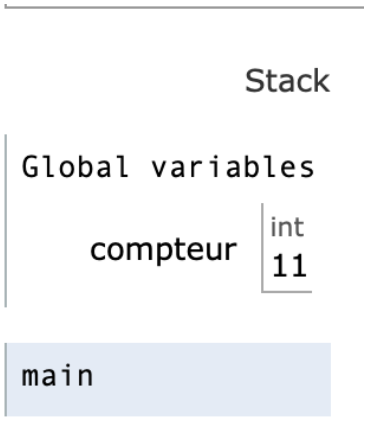


Pas à pas cela donne :

```
1  #include <stdio.h>
2  int compteur=10;
3
4  int main()
5  {
6    compteur ++;
7    printf("Compteur = %d",compteur);
8      return 0;
9  }
```




```
1  #include <stdio.h>
2  int compteur=10;
3
4  int main()
5  {
6    compteur ++;
7    printf("Compteur = %d",compteur);
8      return 0;
9  }
```



La fonction `main()` reconnait la variable `compteur` comme étant globales.

b. Deux variables portant le même nom

Soit le programme :

```
#include <stdio.h>
int compteur=10;

int main()
{
  int compteur = 100;
  compteur ++;
  printf("Compteur = %d",compteur);
    return 0;
}
```

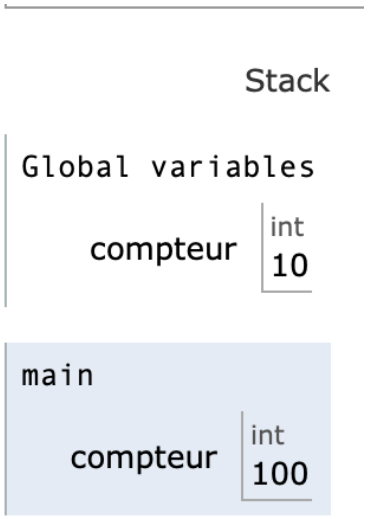
L'affichage est :

```
> make -s
> ./main
Compteur = 101>
```

Explications :

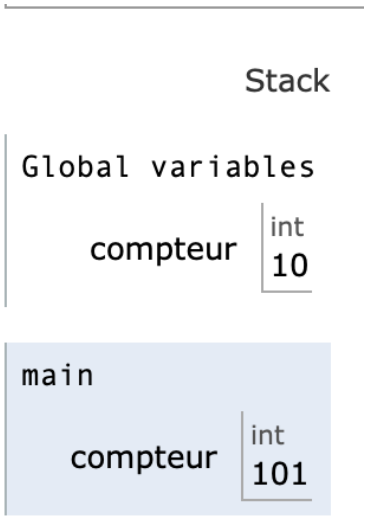
Même si elles portent le même nom, les deux variables ont des espaces mémoires différents : une est globale la seconde est définie à l'intérieur de la fonction `main()` :

```
 1  #include <stdio.h>
 2  int compteur=10;
 3
 4  int main()
 5  {
 6    int compteur = 100;
 7    compteur ++;
 8    printf("Compteur = %d",compteur);
 9      return 0;
10  }
```



Ensuite *compteur++* va incrémenter le compteur du main et pas celle globale :

```
 1  #include <stdio.h>
 2  int compteur=10;
 3
 4  int main()
 5  {
 6    int compteur = 100;
 7    compteur ++;
 8    printf("Compteur = %d",compteur);
 9      return 0;
10  }
```



Et l'affichera donnera Compteur = 101

c. Trois variables avec le même nom

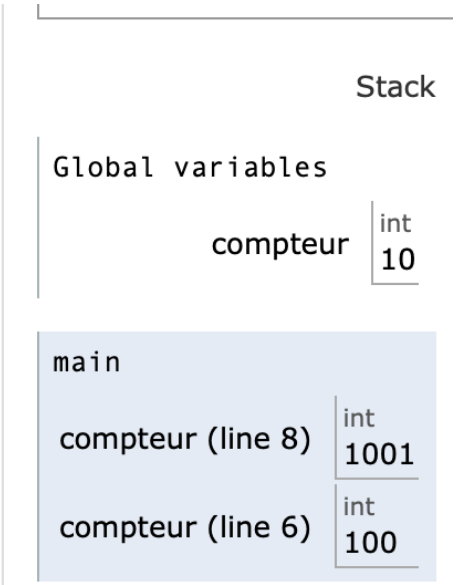
```c
#include <stdio.h>
int compteur=10;

int main()
{
  int compteur = 100;
  {
    int compteur = 1000;
    compteur ++;
    printf("Compteur = %d",compteur);
  }

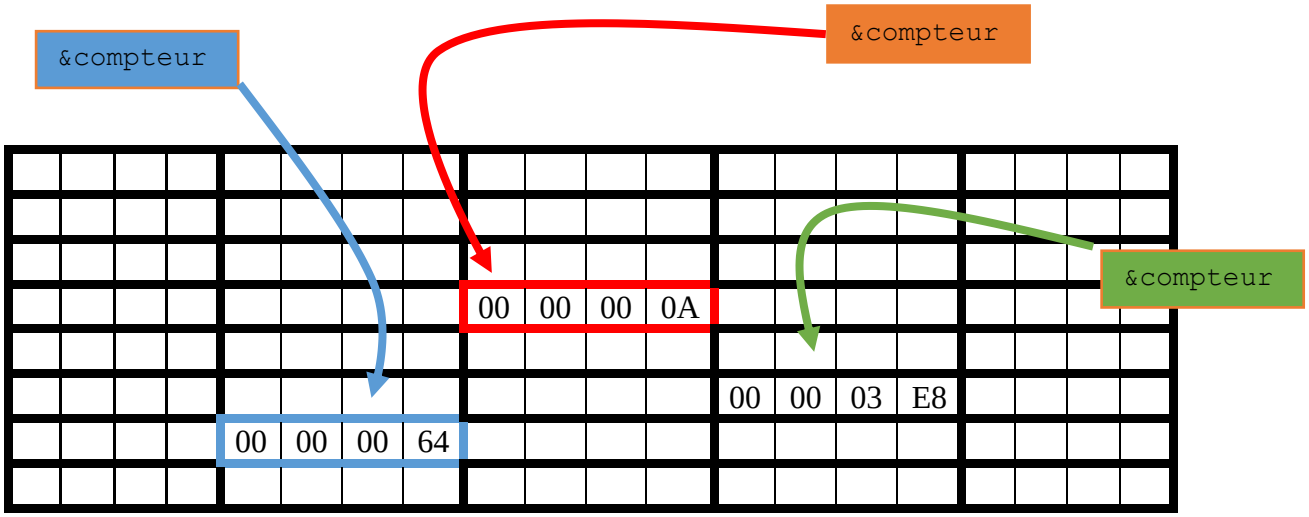
    return 0;
}
```

L'affichera sera
```
make -s
./main
Compteur = 1001
```
car c'est la variable à l'intérieur de l'accolade qui sera prise en compte, celle déclarée à la ligne 8 :

```
1  #include <stdio.h>
2  int compteur=10;
3
4  int main()
5  {
6    int compteur = 100;
7    {
8      int compteur = 1000;
9      compteur ++;
10     printf("Compteur = %d",compteur);
11   }
12
```



En mémoire cela donne quelque chose qui ressemble à cela : trois variables portant le même nom mais à des emplacements mémoire différents.



Mémoire

Rappel : conversion décimale <-> hexadécimale
10 = 0x0A, 100 = 0x64 et 1000 = 0x03E8

d. Dernier exemple :

```
#include <stdio.h>
int compteur=10;

int main()
{
  int compteur = 100;
  {
    int compteur = 1000;
```

```
    compteur ++;
    printf("Compteur = %d\n",compteur);
  }
    printf("Compteur = %d",compteur);

    return 0;
}
```

Quel est l'affichage ?

e. Pour ne pas utiliser de variables globales ?

Cela facilite grandement le travail du développeur !!! Et pourquoi ne pas le faire tout le temps ?

Il y a plusieurs raisons pour cela : la première est une question de mémoire. Tandis que l'espace mémoire d'une variable déclarée dans une fonction est libérée lorsqu'on sort de celle-ci, la mémoire associée à la variable globale est réservée tout le temps que le programme s'exécute. Sur certains systèmes, notamment les systèmes embarqués, si on abuse de ce type de variables on risque d'être à court de mémoire.

Ensuite, encore une fois pour des questions de lisibilité, et d'éviter les bugs sont utilisation est proscrite dans un « réel » projet informatique : vous pouvez travailler à plus d'une dizaine sur le même code, imaginez si tout le monde n'utiliserait que des variables globales, on ne pourrait s'y retrouver. Sans compter la taille mémoire nécessaire.

A titre d'exemple, les codes des systèmes d'exploitation modernes, tels que Windows 11, sont composés de millions de lignes de code.

III. POINTEURS ET ADRESSES

1. *Adresses*

a. Définition d'une adresse

En langage C, une adresse est un nombre qui indique l'emplacement en mémoire d'une variable ou d'une fonction. Chaque variable et chaque fonction possède une adresse unique en mémoire

b. Exemple avec les tableaux

Pour un char :

Un char est codé en mémoire sur un octet et donc pour un tableau de 10 char, l'adresse entre chaque élément est de 1 :

```
  char untableauChar[10];
  printf("la taille d'un emplacement mémoire pour un char est %d
octet\n",sizeof(char));

 for (int i = 0; i < 10; i++)
      printf("L'adresse de @tableauChar+i = %p\n",(void *)(untableauChar+i));

}
```

L'affichage montre que

- La taille d'une variable de type char est de 1 octet (`sizeof(char)`)

```
la taille d'un emplacement mémoire pour un char est 1 octet
```

- que les adresses dans le tableau se suivent :

```
L'adresse de @tableauChar+i = 0x7ffded676a96
L'adresse de @tableauChar+i = 0x7ffded676a97
L'adresse de @tableauChar+i = 0x7ffded676a98
L'adresse de @tableauChar+i = 0x7ffded676a99
```

```
L'adresse de @tableauChar+i = 0x7ffded676a9a
L'adresse de @tableauChar+i = 0x7ffded676a9b
L'adresse de @tableauChar+i = 0x7ffded676a9c
L'adresse de @tableauChar+i = 0x7ffded676a9d
L'adresse de @tableauChar+i = 0x7ffded676a9e
L'adresse de @tableauChar+i = 0x7ffded676a9f
```

Pour un int :

Un int est codé en mémoire sur quatre octets et donc pour un tableau de 10 char, l'adresse entre chaque élément est de 4 :

```
  int unTableauInt[10];
  printf("la taille d'un emplacement mémoire pour un int est %d octets\n",
sizeof(int));
  for (int i = 0; i < 10; i++)
      printf("L'adresse de unTableauInt+i = %p\n",(void *)(unTableauInt+i));
  printf("\n");

}
```

L'affichage montre que

- La taille d'une variable de type char est de 1 octet (`sizeof(char)`)

```
la taille d'un emplacement mémoire pour un char est 4 octets
```

- que les adresses dans le tableau se suivent :

```
L'adresse de unTableauInt+i = 0x7ffded676aa0
L'adresse de unTableauInt+i = 0x7ffded676aa4
L'adresse de unTableauInt+i = 0x7ffded676aa8
L'adresse de unTableauInt+i = 0x7ffded676aac
L'adresse de unTableauInt+i = 0x7ffded676ab0
L'adresse de unTableauInt+i = 0x7ffded676ab4
L'adresse de unTableauInt+i = 0x7ffded676ab8
L'adresse de unTableauInt+i = 0x7ffded676abc
L'adresse de unTableauInt+i = 0x7ffded676ac0
L'adresse de unTableauInt+i = 0x7ffded676ac4
```

Explications du code :

Au lieu d'utiliser `%d` pour afficher les adresses, `%p` a été utilisé. Ce formatage est conçu pour afficher les adresses mémoire. L'expression `(unTableauInt + i)` a été converti en `(void *)` pour correspondre au format `%p`.

2. Pointeurs

Les pointeurs sont des variables qui contiennent une adresse mémoire en tant que valeur. En d'autres termes, ils pointent vers une zone de la mémoire où une valeur est stockée.

a. Définition et déclaration d'un pointeur

En langage C, un pointeur est une variable qui contient l'adresse d'une autre variable.
Un pointeur est défini en utilisant le caractère astérisque (*) qui indique qu'il s'agit d'un pointeur. Par exemple:

```
int *monPointeur;
```

Cette instruction déclare une variable appelée `monPointeur`, qui est un pointeur vers un entier.

b. Initialisation d'un pointeur en langage C

Un pointeur doit être initialisé avant d'être utilisé.
Il est possible d'obtenir l'adresse d'une variable en utilisant l'opérateur " & " suivi du nom de la variable.
Pour initialiser un pointeur, on lui attribue l'adresse d'une variable à l'aide de l'opérateur d'adresse (&). Par exemple :

```
int var = 5;
```

```
int *monPointeur;
monPointeur = &var;
```

Ici, `monPointeur` est initialisé avec l'adresse de la variable var à l'aide de l'opérateur d'adresse (&).
Dans cet exemple, la variable " `monPointeur` " est un pointeur qui contient l'adresse en mémoire de la variable " var ".

Maintenant, `monPointeur` pointe vers l'adresse de la variable var :

c. Utilisation

On peut accéder à la valeur de `var` en utilisant le pointeur `monPointeur`. Par exemple :

```
printf("La valeur de la variable var est : %d", *monPointeur);
```

Ici, `*monPointeur` est utilisé pour accéder à la valeur de la variable `var` à partir de son adresse stockée dans `monPointeur`.

d. Affichage de l'adresse

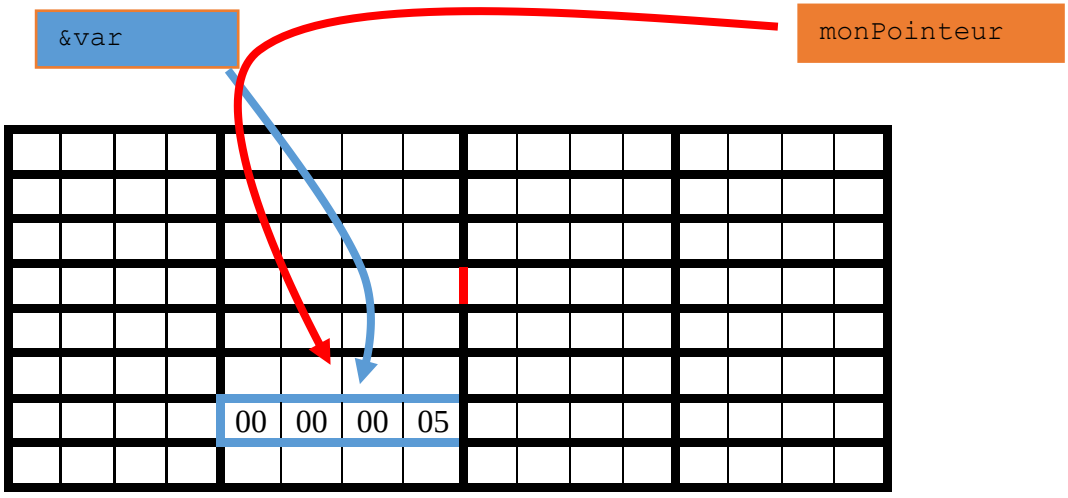
```
int var = 5;
int *monPointeur;

monPointeur = &var; // monPointeur contient une adresse
printf("monPointeur = %p\n", monPointeur);
```

L'affichage nous donne bien l'adresse :

```
monPointeur = 0x7fffc5b05504
```

En mémoire cela donne quelque chose qui ressemble à cela : trois variables portant le même nom mais à des emplacements mémoire différents.



Mémoire

e. Si le pointeur n'est pas initialisé ?

```
int var = 5;
int *monPointeur;

*monPointeur = 10;
printf("monPointeur = %p\n", monPointeur);
```

A l'exécution, le programme se « plante ».

En effet, il n'y a aucune adresse dans la variable pointeur

```
signal: illegal instruction (core dumped)
```

IV. UTILISATION DES POINTEURS

1. Avec les tableaux

Les pointeurs en langage C peuvent être utilisés avec les tableaux pour faciliter l'accès et la manipulation des données. Lorsqu'un tableau est déclaré, il est alloué en mémoire comme un bloc contigu de données. L'adresse de la première case mémoire du tableau peut être stockée dans un pointeur.

a. declaration

Par exemple, pour déclarer un tableau d'entiers de 5 éléments et initialiser un pointeur pour pointer vers le premier élément, on peut utiliser le code suivant :

```
int tableau[5] = {1, 2, 3, 4, 5}; // Déclaration et initialisation du tableau
int *pointeur = tableau; // Initialisation du pointeur
```

Dans cet exemple, le tableau est déclaré avec 5 éléments et initialisé avec des valeurs. Ensuite, un pointeur de type int est déclaré et initialisé avec l'adresse de la première case mémoire du tableau à l'aide de la notation d'adresse &tableau[0].
Cependant, dans ce cas, il est possible d'utiliser simplement le nom du tableau pour obtenir l'adresse du premier élément, car le nom du tableau est équivalent à l'adresse de sa première case mémoire.

Vous en souvenez de
`scanf("%d",&i)` et `scanf("%d",tab)` ?
Et bien la fonction `scanf` demande en paramètre l'adresse de la variable : c'est `&i` pour un entier et le nom du tableau correspond à son adresse, **d'où l'absence de &.**

b. parcours

Il est également possible de parcourir le tableau en utilisant un pointeur et l'opérateur d'incrémentation/decrémentation ++/--. Par exemple, pour afficher les éléments du tableau à l'aide d'un pointeur, on peut utiliser le code suivant :

```
for (int i = 0; i < 5; i++) {
    printf("%d ", *pointeur); // Affichage de la valeur pointée par le pointeur
    pointeur++; // Incrémentation du pointeur pour pointer vers l'élément suivant
}
```

Dans cet exemple, la boucle for parcourt chaque élément du tableau en utilisant le pointeur. À chaque itération, la valeur pointée par le pointeur est affichée à l'aide de l'opérateur *. Ensuite, le pointeur est incrémenté pour pointer vers l'élément suivant à l'aide de l'opérateur ++.

c. Accès direct

L'utilisation des pointeurs avec les tableaux peut également faciliter la manipulation des données en mémoire, en permettant par exemple d'échanger les valeurs de deux éléments du tableau.

Par exemple, pour échanger les valeurs du premier et du dernier élément du tableau, on peut utiliser le code suivant :

```
int temp = *pointeur; // Stockage de la valeur pointée par le pointeur dans une
variable temporaire
*pointeur = *(pointeur + 4); // Remplacement de la valeur pointée par le pointeur par
celle du dernier élément
```

```
*(pointeur + 4) = temp; // Remplacement de la valeur pointée par le pointeur du dernier élément par celle de la variable temporaire
```

Dans cet exemple, la valeur du premier élément du tableau est stockée dans une variable temporaire `temp`. Ensuite, la valeur pointée par le pointeur est remplacée par celle du dernier élément du tableau à l'aide de l'opérateur + pour accéder à l'élément à la position 4. Enfin, la valeur pointée par le pointeur du dernier élément est remplacée par la valeur temporaire stockée précédemment.

2. Les opérations arithmétiques sur les pointeurs avec les tableaux

L'arithmétique des pointeurs en langage C permet de faire des opérations mathématiques sur les adresses mémoires. Elle est très utilisée pour parcourir des tableaux et des structures de données.

Les opérations arithmétiques qui peuvent être effectuées sur des pointeurs incluent :

a. Addition

L'addition permet de déplacer un pointeur vers l'avant (vers des adresses mémoires plus grandes).
Par exemple :

```
int tab[5] = {1, 2, 3, 4, 5};
int *ptr = tab; // ptr pointe sur le premier élément de tab

ptr = ptr + 2; // ptr pointe maintenant sur le troisième élément de tab (3)
```

b. la soustraction

La soustraction : permet de déplacer un pointeur vers l'arrière (vers des adresses mémoires plus petites).
Par exemple :

```
int tab[5] = {1, 2, 3, 4, 5};
int *ptr = &tab[4]; // ptr pointe sur le dernier élément de tab

ptr = ptr - 2; // ptr pointe maintenant sur le troisième élément de tab (3)
```

c. La comparaison

La comparaison : permet de comparer des pointeurs entre eux. Les opérateurs de comparaison disponibles sont <, <=, > et >=.
Par exemple :

```
int tab[5] = {1, 2, 3, 4, 5};
int *ptr1 = &tab[2]; // ptr1 pointe sur le troisième élément de tab (3)
int *ptr2 = &tab[4]; // ptr2 pointe sur le dernier élément de tab (5)

if (ptr1 < ptr2) {
    printf("ptr1 pointe sur une adresse mémoire plus petite que ptr2\n");
}
```

d. Conclusion

Il est important de noter que toutes les opérations arithmétiques sur les pointeurs doivent être effectuées avec prudence, car elles peuvent conduire à des erreurs de segmentation (segmentation fault) si elles sont mal utilisées.

3. A. Pointeurs et fonctions

a. Rappel passage par valeur

C'est l'utilisation que l'on a depuis le début de l'utilisation des fonctions.

Le passage par valeur en langage C implique que lorsqu'une fonction est appelée, les valeurs des paramètres sont copiées dans des variables locales de la fonction. Toute modification apportée à ces variables locales n'affecte pas les variables externes correspondantes. Par conséquent, le passage par valeur est souvent utilisé pour les fonctions qui n'ont pas besoin de modifier les valeurs des variables d'origine.

Voici un exemple :

```c
#include <stdio.h>

void increment(int a) {
    a++;
}

int main() {
    int x = 5;
    increment(x);
    printf("%d", x); // affiche 5, car la fonction n'a pas modifié la valeur de x
    return 0;
}
```

Dans ce code, la fonction *incrémente* prend un paramètre entier a par valeur. La valeur de *x* est passée à la fonction *increment* en tant que paramètre, et la fonction incrémente la valeur de *a* localement. Cependant, la modification de la variable a n'affecte pas la valeur de x en dehors de la fonction *increment*. Ainsi, l'appel à *printf* affiche toujours la valeur de x initiale, soit 5.

En somme, le passage par valeur est utile lorsque l'on veut que les variables d'origine ne soient pas modifiées par une fonction.

b. Passage de pointeurs en paramètres de fonctions

Dans le cas où on désire modifier la valeur de la variable dans la fonction, le passage par adresse est une technique de programmation qui le permet. Pour cela, on passe l'adresse mémoire de la variable en question en paramètre de la fonction. La fonction peut ainsi modifier la valeur de la variable en accédant directement à sa position mémoire.

Voici un exemple d'utilisation du passage par adresse en langage C :

```c
#include <stdio.h>

void incrementer(int *p) { // p est une adresse et *p est le contenu à l'adresse p
    *p = *p + 1;
}

int main() {
    int a = 10;
    printf("La valeur de a avant l'incrémentation est : %d\n", a); // Affiche 10
    incrementer(&a); // On passe l'adresse de a à la fonction
    printf("La valeur de a après l'incrémentation est : %d\n", a); // Affiche 11
    return 0;
}
```

Dans cet exemple, la fonction *incrementer* prend en paramètre un pointeur p vers un entier :

```c
    incrementer(&a); // On passe l'adresse de a à la fonction
```

En accédant à la valeur pointée par p, on peut modifier directement la valeur de la variable passée en paramètre (`a` dans le `main`). On utilise l'opérateur * pour accéder à la valeur pointée par le pointeur.

En passant l'adresse de a à la fonction *incrementer* avec &a, on permet à la fonction de modifier directement la valeur de a.

```
void incrementer(int *p) {
```

Ainsi, après l'appel de la fonction, la valeur de *a* a été incrémentée de 1.

```
    printf("La valeur de a après l'incrémentation est : %d\n", a); // Affiche 11
```

Le passage par adresse est particulièrement utile lorsque l'on souhaite modifier la valeur d'une variable dans une fonction, mais que cette variable n'est pas accessible depuis la fonction. On peut également utiliser le passage par adresse pour optimiser les performances en évitant de copier des structures de données volumineuses en mémoire.

4. Pointeur et tableaux à une dimension

En utilisant des pointeurs avec des tableaux, nous pouvons passer des tableaux en tant qu'arguments de fonctions, ce qui nous permet d'effectuer des opérations sur les éléments des tableaux de manière plus efficace.

a. Tableau à une dimension première version

Voici un tout premier exemple de fonction qui met à une valeur donnée en paramètre tous les éléments d'un tableau dont l'adresse est transmise en argument :

```
void RemplirTableau(int unTableau[10], int valeur) {
  for (int i = 0; i < 10; i++)
    unTableau[i]= valeur;
}
```

L'appel de cette fonction et très simple également : `RemplirTableauV1(tableau);`

Le code complet est :

```
#include <stdio.h>

void RemplirTableau(int unTableau[10], int valeur) {
  for (int i = 0; i < 10; i++)
    unTableau[i]= valeur;
}

int main(void) {
  int tableau[10]; // Tableau de 10 entiers
  RemplirTableau(tableau,10);
  for (int i = 0; i < 10; i++)
    printf("tab[%d] = %d\n", i, tableau[i]);
  return 0;
}
```

La taille tableau et facultatif. En effet le compilateur peux connaître facilement le nombre d'éléments mais cette écriture est à préférer pour des questions de lisibilité : le développeur de la fonction et celui qui va lire le code c'est ainsi que la dimension et de 10.

b. D'autres version de déclaration de fonction possible

Indiquer la dimension dans ce cas n'est pas indispensable il est possible d'écrire l'en-tête de la fonction sans spécifier la dimension :

```
void RemplirTableau(int unTableau[], int valeur) {
```

ou encore en utilisant un pointeur sur le tableau. :

```
void RemplirTableau(int *unTableau, int valeur) {
```

Et l'appel est identique

```
  RemplirTableau(tableau,10);
```

c. D'autres version du code de la fonction :

Il est possible d'écrire le code de plusieurs d'autres façons :

Version itération i et pointeur

On peut même jouer avec les adresses dans l'exemple suivant on utilise un pointeur sur un entier dans la boucle on et initialise la valeur de ce pointeur et on va jusqu'à l'adresse du début du tableau +10 saut dentier en mémoire. on incremente la mémoire p de 1. si les nombres entiers sont codés sur 4 octets, p++ équivaut a augmenté p qui est une adresse en mémoire de 4 .

```
void RemplirTableau(int *unTableau, int valeur) {
  for (int i = 0; i < 10; i++)
    *(unTableau + i) = valeur;
}
```

Explications :

1. La variable "unTableau" est initialisée avec l'adresse du tableau fournie en paramètre.
2. À chaque itération, l'adresse du tableau est augmenté de 1 et la valeur est affectée à l'adresse mémoire pointée par `unTableau + i`. `*(unTableau+i)` est le contenu de la mémoire pointée par `unTableau + i`

Version itération i et pointeur

De la même façon le corps de la fonction écris précédemment est le plus simple à comprendre. Cependant il est est possible d'écrire comme ceci où *(t+i) signifie l'emplacement à l'adresse plus I

```
void RemplirTableau(int *unTableau, int valeur) {
  for (int i = 0; i < 10; i++, unTableau++)
    *unTableau = valeur;
}
```

Explications :

1. La variable "unTableau" est initialisée avec l'adresse du tableau fournie en paramètre.
2. Le compteur d'itération "i" est utilisé pour suivre l'index actuel dans le tableau, tandis que l'expression "unTableau++" est utilisée pour avancer le pointeur vers l'élément suivant dans le tableau à chaque itération.
3. À chaque itération, la valeur est affectée à l'adresse mémoire pointée par le pointeur `unTableau`. `*unTableau` est le contenu de la mémoire pointée par `unTableau`

d. Récapitulatif

En conclusion et souvent c'est souvent le cas en langage C il est possible décrire du code simple ou de s'amuser avec les nombreuses possibilités du C

Pour l'entête : trois possibilités

```
1. void RemplirTableau(int unTableau[10], int valeur) {
2. void RemplirTableau(int unTableau[], int valeur) {
3. void RemplirTableau(int *unTableau, int valeur) {
```

Et l'appel est identique

```
RemplirTableau(tableau,10);
```

Pour le corps de la fonction : trois possibilités

```
1. unTableau[i]= valeur;
2. *(unTableau + i) = valeur;
3. *unTableau = valeur;
```

e. Nombre d'éléments

En langage C il n'existe aucune fonction permettant de connaitre le nombre d'éléments dans un tableau. Exception faite des chaînes de caractères où le dernier caractère signifie fin de chaîne.

5. Tableaux à plusieurs dimensions

Les tableaux multidimensionnels sont passés aux fonctions sous forme de pointeurs. Voici un exemple de l'utilisation de pointeurs avec un tableau à deux dimensions (matrice) dans une fonction :

```c
#include <stdio.h>

void print_matrice(int rows, int cols, int (*matrice)[cols]) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%d ", matrice[i][j]);
        }
        printf("\n");
    }
}

int main() {
    int matrice[3][4] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };

    print_matrice(3, 4, matrice);

    return 0;
}
```

Dans cet exemple, nous avons défini une fonction `print_matrice` qui prend en paramètre les dimensions de la `matrice` (`rows` et `cols`) et un pointeur vers un tableau de la taille `cols`. Le pointeur vers le tableau est déclaré avec la syntaxe `int (*matrice)[cols]`. La fonction utilise ensuite ces paramètres pour afficher la matrice.

De même, pour un tableau à trois dimensions, voici un exemple :

```c
#include <stdio.h>

void print_tableau3D(int dim1, int dim2, int dim3, int (*tableau3D)[dim2][dim3]) {
    for (int i = 0; i < dim1; i++) {
        for (int j = 0; j < dim2; j++) {
            for (int k = 0; k < dim3; k++) {
                printf("%d ", tableau3D[i][j][k]);
            }
            printf("\n");
        }
        printf("\n");
    }
}

int main() {
    int tableau3D[2][3][4] = {
        {
            {1, 2, 3, 4},
            {5, 6, 7, 8},
            {9, 10, 11, 12}
        },
        {
            {13, 14, 15, 16},
```

```
            {17, 18, 19, 20},
            {21, 22, 23, 24}
        }
    };

    print_tableau3D(2, 3, 4, tableau3D);

    return 0;
}
```

Dans cet exemple, la fonction `print_tableau3D` prend en paramètre les dimensions du tableau à trois dimensions (`dim1, dim2 et dim3`) et un pointeur vers un tableau de taille `dim2xdim3`. Le pointeur vers le tableau est déclaré avec la syntaxe `int (*tableau3D)[dim2][dim3]`. La fonction utilise ensuite ces paramètres pour afficher le tableau à trois dimensions.

6. *Passage de structure*

En passant un pointeur vers une structure plutôt que la structure elle-même, vous évitez de copier l'intégralité de la structure (ce qui peut être coûteux en termes de mémoire et de temps d'exécution) et vous permettez à la fonction de modifier directement la structure d'origine si nécessaire.

Voici un exemple détaillé du passage d'une structure par pointeur dans une fonction :

```
#include <stdio.h>

// Définition de la structure
typedef struct {
    char nom[50];
    int age;
    float salaire;
} Employe;

// Fonction qui prend un pointeur vers une structure Employe et modifie ses propriétés
void augmenter_salaire(Employe *employe, float pourcentage) {
    employe->age += 1;
    employe->salaire *= (1.0 + pourcentage / 100.0);
}

// Fonction qui affiche les informations d'un employé
void afficher_employe(Employe *employe) {
    printf("Nom: %s\n", employe->nom);
    printf("Age: %d\n", employe->age);
    printf("Salaire: %.2f\n", employe->salaire);
}

int main() {
    Employe e1 = {"Alice", 30, 5000.0};

    printf("Avant l'augmentation :\n");
    afficher_employe(&e1);

    augmenter_salaire(&e1, 10.0);

    printf("\nApres l'augmentation :\n");
    afficher_employe(&e1);

    return 0;
}
```

Dans cet exemple :

1. Nous définissons une structure `Employe` avec trois champs : `nom`, `age` et `salaire`.

2. Nous créons une fonction `augmenter_salaire` qui prend un pointeur vers une structure `Employe` et un pourcentage d'augmentation de salaire. La fonction modifie directement les propriétés de la structure pointée en utilisant l'opérateur `->`.
3. Nous créons une fonction `afficher_employe` qui prend un pointeur vers une structure `Employe` et affiche ses propriétés.
4. Dans la fonction `main`, nous créons une instance de la structure `Employe` appelée `e1` et initialisons ses champs.
5. Nous appelons la fonction `afficher_employe` pour afficher les informations de l'employé avant l'augmentation de salaire en passant l'adresse de la structure `e1` en utilisant l'opérateur `&`.
6. Nous appelons la fonction `augmenter_salaire` pour augmenter l'âge et le salaire de l'employé en passant l'adresse de la structure `e1`.
7. Nous appelons à nouveau la fonction `afficher_employe` pour afficher les informations de l'employé après l'augmentation de salaire.

Le passage de la structure par pointeur permet à la fonction `augmenter_salaire` de modifier directement les propriétés de la structure `e1`, sans avoir à copier l'intégralité de la structure dans un nouvel espace mémoire.

V. ALLOCATION DYNAMIQUE DE MEMOIRE

L'allocation dynamique de mémoire en langage C permet de réserver de la mémoire à l'exécution d'un programme, plutôt qu'à la compilation. Cela offre une plus grande flexibilité pour gérer les ressources mémoire. La fonction `malloc()` est l'une des fonctions de la bibliothèque standard C qui permet d'allouer de la mémoire de manière dynamique.

1. Présentation de malloc()

1. `malloc()` est une fonction de la bibliothèque `stdlib.h`.
2. Syntaxe : `void* malloc(size_t size);`
3. La fonction `malloc()` prend en paramètre la taille en octets de la mémoire à allouer.
4. Elle renvoie un pointeur de type `void*` vers l'espace mémoire alloué.
5. Si l'allocation échoue, elle renvoie un pointeur `NULL`.

2. Exemple d'utilisation de malloc() pour allouer un tableau d'entiers

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n, i, *arr;

    printf("Entrez la taille du tableau: ");
    scanf("%d", &n);

    // Allouer de la mémoire pour le tableau
    arr = (int*) malloc(n * sizeof(int));

    // Vérifier si l'allocation a réussi
    if (arr == NULL) {
        printf("Echec de l'allocation de mémoire.\n");
        return 1;
    }

    // Remplir le tableau avec des valeurs
    for (i = 0; i < n; i++) {
        arr[i] = i + 1;
```

```
    }

    // Afficher le tableau
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    // Libérer la mémoire allouée
    free(arr);

    return 0;
}
```

Dans cet exemple, nous allouons de la mémoire pour un tableau d'entiers dont la taille est déterminée par l'utilisateur. Nous utilisons `malloc()` pour allouer la mémoire nécessaire et vérifions si l'allocation a réussi. Ensuite, nous remplissons le tableau avec des valeurs et affichons son contenu. Finalement, nous libérons la mémoire allouée avec la fonction `free()`.

3. Utilisation de malloc() pour allouer de la mémoire à une structure

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int x;
    int y;
} Point;

int main() {
    Point *p;

    // Allouer de la mémoire pour la structure Point
    p = (Point*) malloc(sizeof(Point));

    // Vérifier si l'allocation a réussi
    if (p == NULL) {
        printf("Echec de l'allocation de mémoire.\n");
        return 1;
    }

    // Attribuer des valeurs aux champs de la structure
    p->x = 10;
    p->y = 20;

    // Afficher les valeurs des champs de la structure
    printf("Point: (%d, %d)\n", p->x, p->y);

    // Libérer la mémoire allouée
    free(p);

    return 0;
}
```

Dans cet exemple, nous utilisons `malloc()` pour allouer de la mémoire à une structure